

EBOOK

DEVOPS E TESTES AUTOMATIZADOS





Sumário

1	Introdução à <i>DevOps</i>	5
1.1	O que é <i>DevOps</i> ?	5
1.2	História do Movimento <i>DevOps</i>	8
1.2.1	Movimentos Influenciadores	8
1.3	Filosofia <i>DevOps</i>	9
1.3.1	Benefícios do <i>DevOps</i>	10
1.4	Práticas <i>DevOps</i>	11
1.5	Ciclo <i>DevOps</i>	13
1.6	Conclusões	14
2	Integração Contínua	16
2.1	Integração Contínua	16
2.1.1	Componentes da Integração Contínua	17
2.2	Práticas da Integração Contínua	17
2.3	Sistemas de Controle de Versão	20
2.3.1	Sistemas Locais de Controle de Versão	21
2.3.2	Sistemas Centralizados de Controle de Versão	21
2.3.3	Sistemas Distribuídos de Controle de Versão	21
2.4	Automação de <i>Build</i>	22
2.4.1	Make	22
2.4.2	Apache Ant	23
2.4.3	Apache Maven	25

2.4.4	Gradle	27
2.5	Sistemas de Integração Contínua	28
2.6	Conclusões	29
3	DevOps: Modelos de Maturidade, Estatísticas e Ferramentas	30
3.1	Modelos de Maturidade	30
3.1.1	Modelo de Maturidade <i>DevOps</i> - Atlassian	31
3.1.2	Modelo de Maturidade da Entrega Contínua	32
3.2	Estatísticas <i>DevOps</i>	34
3.3	Ferramentas <i>DevOps</i>	36
3.4	Estudo de Caso - Netflix	39
3.5	Conclusões	40
4	Testes: Conceitos relevantes	41
4.1	Abordagens, Modalidades e Tipos de Teste	43
4.1.1	Abordagem Caixa Branca	43
4.1.2	Abordagem Caixa Preta	43
4.1.3	Abordagem de Regressão	43
4.1.4	Testes Funcionais	44
4.1.5	Testes Não funcionais	44
4.1.6	Teste End-to-end	44
4.1.7	Análise de Mutantes	44
4.1.8	Teste Unitário	44
4.1.9	Teste Integrado	44
4.1.10	Testes de Aceitação do Usuário	44
4.1.11	Smoke Testing	45
4.1.12	Testes Exploratórios	45
4.1.13	TDD - Test Driven Development	45
4.1.14	BDD - Behaviour Driven Development	45
4.1.15	Especificações Executáveis	46
4.1.16	ATDD - Acceptance Test Driven Development	46
4.1.17	Teste A/B	46
4.2	Casos de Falhas em Software	46
4.3	Conclusões	47
5	Testes Automatizados	48
5.1	Ferramentas de Testes Automatizados	49
5.2	Estudo de Caso: Teste Automatizado em uma Equipe Agile	49
5.3	Estudo de Caso: Google GWS	54

5.4	Provas de Conceito de Testes Automatizados	57
5.4.1	JUnit utilizando TDD	57
5.4.2	BDD e Smoke Test utilizando Cucumber e Selenium	60
5.5	Conclusões	65
	Referências	67
	Sobre o Professor	71
	Sobre o Professor	72



A Disciplina de *DevOps* Testes Automatizados está profundamente relacionada com as demais disciplinas da *Pós-Graduação EAD em Desenvolvimento Orientado a Objetos em Java*, em especial às disciplinas de *Metodologias Ágeis* e *Padrões de Projeto e Qualidade do Código Fonte*, *Gerência de Configuração*, *Controle de Versão e Mudanças*, *Microserviços em Java* e *Containers: Docker e Kubernetes*. Desta forma, é fundamental que o estudante consiga relacionar os conceitos aprendidos nas demais disciplinas com os conceitos aqui apresentados.

Este capítulo tem por objetivo introduzir aos conteúdos relacionados à *DevOps & Testes Automatizados*, que serão abordados com maior profundidade nos capítulos seguintes. Ao término deste capítulo é esperado que o estudante consiga responder às seguintes perguntas:

- O que é *DevOps*?
- Como o movimento *DevOps* surgiu?
- O que influenciou o surgimento do movimento *DevOps*?
- Quais são as principais práticas *DevOps*?
- Quais as principais vantagens da utilização das práticas *DevOps*?
- Quais são as etapas do ciclo *DevOps*?

1.1 O que é *DevOps*?

DevOps pode ser tratado como um movimento cultural ou mesmo uma filosofia. Sendo definido como um conjunto de práticas que visam integrar as equipes de **desenvolvimento** de software e de **operações**, bem como equipes de apoio como **garantia de qualidade** de software e **segurança da informação** (KIM et al., 2016). A Figura 1.1 traz um diagrama ilustrando as intersecções entre estas diferentes áreas de atuação. Nem todas as organizações são estruturadas exatamente desta maneira, porém as práticas do *DevOps* se situam justamente na intersecção entre estas atividades.

O termo *DevOps* trata-se de uma união das palavras: *Development* (Desenvolvimento) e *Operations* (Operações). Também existe o termo *DevSecOps*, que foi criado para enfatizar a necessidade da inclusão área segurança da informação como uma responsabilidade compartilhada e integrada do início ao fim. Este termo, inclui a palavra *Information Security – InfoSec* (Segurança da

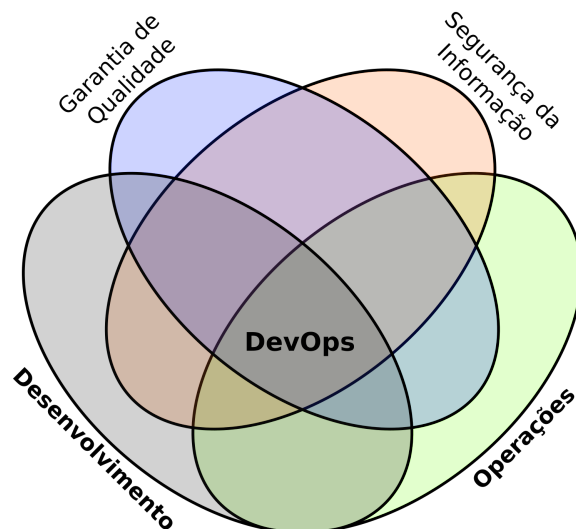


Figura 1.1: Intersecções entre as áreas de Desenvolvimento, Operações, Segurança da Informação e Garantia de Qualidade.

informação). Independente de receber o nome de *DevOps* ou *DevSecOps*, o ideal é sempre incluir a segurança e integrá-la durante todo o ciclo de vida do software (RED HAT, INC, 2019).

As equipes de desenvolvimento e operações tradicionalmente atuam de maneira independentes umas das outras, muitas vezes operando de maneira totalmente caótica e não coordenada. A área de desenvolvimento usualmente busca valorizar o produto da empresa atendendo às expectativas dos clientes por novos recursos. Enquanto que a área de operações deseja o mínimo de alterações no ambiente de produção que podem causar instabilidades e desvalorizar o produto da empresa.

Desta forma os interesses das equipes podem ser contraditórios, enquanto uma equipe quer evoluir (desenvolvimento), a outra quer continuar garantindo os serviços (operações). Além disto, estas equipes usualmente costumam possuir lideranças distintas, bem como métricas e indicadores de desempenho diferentes. Estes conflitos inerentes entre desenvolvimento e operações acabam criando uma espiral descendente que resulta em um tempo cada vez maior para os novos produtos e recursos chegarem ao mercado, e também reduzindo a qualidade e aumentando a quantidade de *débito técnico* (KIM et al., 2016).

Como resultado disto as equipes não conseguem cumprir com seus objetivos, e a organização como um todo passa a ficar insatisfeita com o desempenho dos times de TI. Isto resulta em cortes de orçamento, frustrações e profissionais infelizes que se sentem impotentes (KIM et al., 2016).

A forma como as organizações são estruturadas e seus mecanismos de comunicação internos impactam fortemente os sistemas que elas criam. Isto levou a criação da chamada *Lei de Conway* (NEWMAN, 2015):

“Qualquer empresa que projeta um sistema (definição mais ampla aqui do que apenas sistemas de informação), inevitavelmente produz um projeto cuja estrutura é uma cópia da estrutura de comunicação da organização.” – Melvin Conway

O movimento *DevOps* começou a tomar forma pelas comunidades de desenvolvimento de software e operações de TI. As comunidades já estavam insatisfeitas com este modelo de trabalho e passaram a se expressar contra ele, acreditando que com certeza existiria uma maneira melhor

de desempenharem seus trabalhos. As práticas *DevOps* nos permitem solucionar estes tipos de problemas, além de simultaneamente melhorar o desempenho da organização como um todo, atingindo os objetivos das diversas funções e melhorando as condições de trabalho (KIM et al., 2016).

A filosofia do *DevOps* prega uma maneira distinta de pensar as diversas atividades e a colaboração entre profissionais envolvidos no ciclo de vida de desenvolvimento de software, tornando o ambiente mais colaborativo e proporcionando maior qualidade e agilidade nas entregas. Com o *DevOps* as barreiras, sejam elas físicas ou culturais que estejam impedindo a comunicação entre as equipes devem ser removidas, assim as equipes podem trabalhar em conjunto e não mais de forma separada.

Desta forma obtém-se uma melhoria na comunicação e a colaboração entre as equipes, e por consequência, um ganho de produtividade dos desenvolvedores e maior confiabilidade para as operações. Tudo isto, impacta de maneira direta e indireta a eficiência e qualidade dos serviços oferecidos para o cliente (AWS DEVOPS, 2019).

Em uma empresa empregando o modelo *DevOps*, os profissionais supervisionam o ciclo completo de desenvolvimento e infraestrutura como parte de suas atribuições (AWS DEVOPS, 2019). Algumas vezes, dependendo da estrutura da empresa, estas equipes podem inclusive ser combinadas em uma única equipe onde os membros do time desempenham mais de uma função, atuando em todo o ciclo de vida do software.

As organizações que adotam os princípios e práticas do *DevOps* conseguem realizar a implantação de alterações com uma frequência muito maior. Em uma era onde as vantagens competitivas requerem um menor tempo para atingir os mercados, as organizações que não conseguem replicar esses princípios e práticas estão predestinadas a perderem mercado para seus competidores (KIM et al., 2016).

Através da criação de ciclos rápidos, todos podem ver os efeitos de suas ações imediatamente. A partir do momento em que as alterações são enviadas ao sistema de controle de versão, testes automatizados podem executar em ambientes semelhantes ao de produção. Isto garante que continuamente tanto código quanto ambiente, possam operar conforme projetado, sempre mantendo em um estado seguro e que possa ser colocado em produção (KIM et al., 2016).

Através da automatização de tarefas repetitivas passíveis de erros e muitas vezes tediosas, as equipes podem garantir maior confiabilidade e consistência na execução, reduzindo o esforço manual empregado. Apesar do movimento *DevOps* apostar em automação para auxiliar as equipes a serem mais eficientes, não trata-se apenas de automação, o movimento vai muito mais além disto (KIM et al., 2016). Como pontuou Christopher Little, um dos primeiros escritores sobre o assunto de *DevOps*:

“*DevOps* não se trata apenas de automação assim como a astronomia não se trata apenas de telescópios.” – Christopher Little

O movimento tem por objetivo melhorar a forma como as pessoas trabalham em conjunto. Obviamente que automatizar uma tarefa repetitiva existente, liberando um profissional de ter de executá-la manualmente e poupando tempo que pode ser gasto com tarefas mais interessantes é benéfico. Porém deve ser considerado o esforço empregado para a automação da tarefa, o tempo necessário para sua execução e frequência que ela é executada (DAVIS; DANIELS, 2016).

1.2 História do Movimento DevOps

Para melhor entender o potencial da revolução trazida pelo *DevOps*, podemos fazer um paralelo com a revolução na indústria da manufatura trazida pela adoção dos princípios e práticas *Lean*. Através da adoção destas práticas, as organizações obtiveram um grande aumento em produtividade, qualidade de produtos e satisfação dos clientes (KIM et al., 2016).

Na área de desenvolvimento de software também houveram grandes revoluções. Com a adoção dos princípios e práticas de *desenvolvimento ágil* a partir do ano 2000, o tempo necessário para desenvolver novas funcionalidades diminuiu drasticamente, enquanto que o tempo necessário para colocá-las em produção ainda continuava praticamente o mesmo (KIM et al., 2016).

Na conferência *Agile* realizada em Toronto em 2008, começou a ser utilizado o termo *infraestrutura ágil* (DAVIS; DANIELS, 2016). Na mesma conferência também houveram debates em torno do tema, a incluindo a apresentação do artigo “*Agile Operations and Infrastructure: How Infra-gile are You?*” (DEBOIS, 2008), por *Patrick Debois*, focado na adoção de metodologias ágeis como o *scrum* na área de operações, trazendo três estudos de caso da aplicação de metodologias ágeis em projetos de infra-estrutura.

Em 2009, na conferência *Velocity*, realizada em *San Jose*, Califórnia, *John Allspaw* e *Paul Hammond* realizaram a apresentação “*10 Deploys per Day: Dev and Ops Cooperation at Flickr*”. Nesta apresentação eles descreveram como conseguiram criar metas compartilhadas entre operações e desenvolvimento, e como utilizaram práticas de integração contínua para tornar as implantações parte da rotina diária das equipes (KIM et al., 2016).

Apesar de não estar presente na conferência, *Patrick Debois* ficou bastante entusiasmado com as ideias da apresentação. Assim ele resolveu criar o primeiro *DevOpsDays*, realizado na Bélgica, criando o termo *DevOps* (KIM et al., 2016). Hoje em dia, o evento é organizado em todo o mundo, sendo promovido por voluntários locais. No Brasil, em 2019 o evento será realizado em cidades como São Paulo-SP, Porto Alegre-RS, Aracaju-SE e Natal-RN ¹.

A partir de 2010, com a introdução do *DevOps*, novas funcionalidades (e até mesmo *startups* inteiras) podem ser criadas em semanas e rapidamente serem implantadas em produção. Isto foi bastante facilitado, pois recursos como hardware, software e mais recentemente computação em nuvem, estão passando a ser tratados como *commodities* (KIM et al., 2016).

1.2.1 Movimentos Influenciadores

O movimento *DevOps* representa a convergência de diversas filosofias e movimentos de gerenciamento que herda seus conceitos de movimentos como o *Sistema Toyota de Produção*, o *Lean*, o *Manifesto Ágil*, entre outros (KIM et al., 2016). Devido a estas características, o movimento propõe a adoção de práticas de automação e monitoramento em todas as fases do ciclo de vida de um software.

O Sistema Toyota de Produção

O *Sistema Toyota de Produção*, também chamado de *Toyotismo*, foi criado por *Taiichi Ohno*, um engenheiro industrial da Toyota. Este sistema contribuiu para a transformação da empresa em uma das principais do segmento automotivo (SITEWARE, 2019).

A ideia principal deste sistema é aumentar a eficiência, a produtividade e a qualidade dos produtos através da redução de desperdícios, e por consequência os custos. Eliminando assim as atividades que não agregam valor para o processo produtivo. Para isto, são utilizados cartões de sinalização que deram origem ao *Kanban*.

¹Mais informações em: <https://www.devopsdays.org/>.

Os princípios fundamentais do sistema são:

- Melhoria Contínua: visão a longo prazo, inovação e procura pela causa dos problemas.
- Respeito às pessoas: confiança mútua e trabalho em equipe.

O Movimento Lean

O sistema de manufatura enxuta (*Lean manufacturing*), tem seus conceitos baseados no Sistema Toyota de Produção, porém expandindo para outras áreas de manufatura.

Os princípios do *Lean* focam na criação de valor através da constante criação de valor aos clientes, adoção do pensamento científico, garantia da qualidade na fonte, liderança com humildade e respeito aos indivíduos (KIM et al., 2016).

Manifesto Ágil

O Manifesto Ágil (*Agile Manifesto*) traz a declaração de 12 princípios fundamentais ao desenvolvimento ágil de software. O manifesto foi criado em 2001, por representantes de diversas metodologias como a Programação Extrema (*eXtreme Programming – XP*), *SCRUM*, Programação Pragmática (*Pragmatic Programming*), entre outras (AGILE ALLIANCE, 2001).

Em essência, podemos sintetizar princípios do manifesto ágil da seguinte forma:

“ Indivíduos e interações mais que processos e ferramentas
Software em funcionamento mais que documentação abrangente
Colaboração com o cliente mais que negociação de contratos
Responder a mudanças mais que seguir um plano

Os itens à direita também possuem o seu valor, porém o movimento ágil busca valorizar mais os itens à esquerda” (AGILE ALLIANCE, 2001).

1.3 Filosofia DevOps

A filosofia do *DevOps* é baseada em 5 pilares conhecidos como estrutura **CALMS**, um acrônimo para *Culture* (Cultura), *Automation* (Automação), *Lean* (Enxuto), *Measurement* (Medição) e *Sharing* (Compartilhamento).

Cultura

A cultura é o ponto mais importante da filosofia *DevOps*, sem a cultura, as práticas do *DevOps* tornam-se algo sem propósito. Ter à disposição diversas ferramentas, automatizar tarefas repetitivas, ou adotar práticas isoladas, podem trazer algum benefício. No entanto, tudo isto torna-se inútil caso não seja acompanhado por um ambiente de trabalho colaborativo e uma maior satisfação dos clientes (ATLASSIAN, 2019).

O *DevOps* incentiva a experimentação e mudanças como uma forma de os profissionais possam continuar evoluindo constantemente as formas de realizarem seus trabalhos. A cultura do *DevOps* prega uma mudança de comportamento, onde toda análise de incidentes é focada na correção dos processos ao invés da identificação dos possíveis culpados, pois o foco do *DevOps* é nas pessoas e nas relações entre elas e não em ferramentas.

Automação

A automação de trabalhos manuais repetitivos produz sistemas mais confiáveis, reduzindo a possibilidade de erros e trazendo maior repetibilidade aos processos. A automação de processos não é algo trivial de ser realizado, mas os benefícios trazidos são muitos (ATLASSIAN, 2019).

Através da automação, é possível obter melhorias em diversas etapas, como por exemplo, compilação, testes e na liberação do software. Através da automação de processos é possível detectar erros e falhas mais cedo, permitindo que sejam corrigidos de maneira mais fácil, reduzindo surpresas na hora da liberação (ATLASSIAN, 2019). Algumas práticas como *integração contínua*, *infraestrutura como código* e *testes automatizados* se baseiam neste conceito.

Lean

A filosofia *DevOps* importa alguns conceitos do *Lean*, especialmente os conceitos de *melhoria contínua* e *aceitação da falha*. Uma mentalidade *DevOps* está constantemente buscando oportunidades de melhoria contínua em toda parte. Cometer eventuais falhas é algo inevitável, mas ao aprender com elas percebemos que o processo de melhoria contínua e a falha caminham lado a lado (ATLASSIAN, 2019). Então as equipes devem estar preparadas para lidar com a falha da melhor maneira possível, enxergando-as como uma forma de melhorar continuamente.

Podemos incorporar também outros conceitos, como a eliminação de desperdícios, eliminando atividades que não agregam valor ao produto ou serviço oferecido para o cliente. Desta forma, podemos manter as coisas de maneira mais simples, sempre mantendo o foco no usuário final.

Medição

Através da medição é possível coletar dados sobre o desempenho, permitindo que se possa saber se os esforços empregados em melhoria contínua estão de fato rendendo frutos. Os dados obtidos pelas medições podem auxiliar as equipes a tomar decisões e inclusive podem ser compartilhados com outras equipes e departamentos da empresa (ATLASSIAN, 2019).

Apesar de podermos definir métricas para quantificar praticamente qualquer coisa, isto não significa que devemos medir tudo. O importante é primeiramente definir um conjunto de algumas métricas básicas, mas que consigam medir os elementos mais importantes do processo e do negócio. Desta forma torna-se mais fácil ir evoluindo para métricas mais sofisticadas, utilizando as informações obtidas para realizar ajustes no processo.

Através da monitoração contínua é possível saber quando um incidente ocorreu, o que permite agir mais rapidamente. Também é possível identificar em que ponto e como a falha ocorreu, o que facilita na realização de ajustes no processo.

Compartilhamento

O *DevOps* também se sustenta no pilar de compartilhamento de informações entre os profissionais das diversas equipes. Com maior integração e colaboração entre profissionais de equipes distintas e com bases de conhecimento diversificadas, é possível encontrar novas formas mais eficientes de desempenhar as tarefas.

1.3.1 Benefícios do DevOps

Removendo-se as barreiras entre as equipes, obtém-se uma melhoria na comunicação entre os profissionais, trazendo mais transparência ao processo e um *feedback* mais rápido. Desta forma, os profissionais passam a ter uma visão mais completa do processo como um todo. Os profissionais também passam a ter ciência da maneira como suas ações afetam não apenas sua equipe, mas também as demais. Assim, o resultado é uma maior colaboração e um maior senso de responsabilidade, que é compartilhado pelos membros de todas as equipes. O *feedback* mais rápido também poupa tempo e reduz ineficiências no processo (ATLASSIAN, 2019; AWS DEVOPS, 2019).

A transparência e melhor comunicação entre as equipes de Desenvolvimento e Operações

possibilita que elas ataquem os problemas de maneira mais ágil. Desta forma, os problemas mais críticos podem ser solucionados em um menor tempo, evitando a redução nos níveis de satisfação dos clientes (ATLASSIAN, 2019).

Com a utilização de ferramentas e processos padronizados, e também pela automação de tarefas as equipes podem realizar liberações de software com mais frequência, diminuindo contratempos e aumentando a produtividade e eficiência das equipes (ATLASSIAN, 2019).

Através de liberações mais frequentes, é possível adaptar-se melhor às necessidades dos clientes e do mercado. Liberações mais frequentes também causam impactos menores, o que gera uma maior estabilidade e qualidade aos processos, tornando-os mais confiáveis. As práticas como integração e entrega contínuas para garantir que todas as alterações sejam seguras e estejam funcionando, enquanto que as práticas de monitoramento e registro em *log* ajudam a fornecer indicadores sobre o desempenho em tempo real (AWS DEVOPS, 2019).

A maioria das equipes enfrentam uma realidade de ter que lidar com trabalhos não planejados. Como o estabelecimento de processos e priorização bem definidas, as equipes podem lidar melhor com este tipo de problema, enquanto continuam focando no trabalho planejado. Fazer a transição e priorização de trabalho não planejado entre diferentes equipes pode ser trabalhoso. Porém com maior visibilidade e proatividade, as equipes podem antecipar e compartilhar melhor os trabalhos não planejados (ATLASSIAN, 2019).

A automação e constância auxiliam no gerenciamento de sistemas complexos e dinâmicos com eficiência e reduzindo riscos. A adoção de práticas como a infraestrutura como código, também auxilia o gerenciamento de ambientes de implantação, teste e produção com maior repetibilidade e eficiência (AWS DEVOPS, 2019). Tudo isto facilita a escalabilidade das soluções, permitindo que novos ambientes possam ser provisionados de maneira pragmática.

Com a definição de políticas de segurança, controles granulares e técnicas de gerenciamento de configuração é possível manter o controle, preservando a conformidade. A definição de políticas de conformidade como código, permite o rastreamento de conformidade de maneira automática e em escala (AWS DEVOPS, 2019).

1.4 Práticas DevOps

Existem algumas práticas que ajudam as organizações a inovarem mais rapidamente, simplificando e automatizando os processos de desenvolvimento de software e gerenciamento de infraestrutura. Uma prática fundamental é a realização de entregas menores, de natureza incrementais, realizando-as de maneira mais frequentes (AWS DEVOPS, 2019).

As organizações também podem se beneficiar de práticas como *microserviços*, *integração contínua*, *entrega contínua*, *infraestrutura como código*. Estas práticas quando aplicadas em conjunto, podem trazer inúmeros benefícios para as organizações.

Desenvolvimento Ágil

Para garantir entregas rápidas e contínuas é fundamental a adoção de metodologia ágil. Desta forma, o desenvolvimento de software deve ser pautado pelos princípios do manifesto ágil, adotando alguma metodologia ágil existente.

Integração Contínua

Através da integração contínua, o fluxo de desenvolvimento de software como um todo pode ser integrado. Assim após qualquer mudança de código disponibilizada no repositório, a compilação

pode ser realizada de maneira automatizada, em seguida testes automatizados podem ser executados, etc. Isto permite encontrar defeitos com maior rapidez, melhorando a qualidade do software (AWS DEVOPS, 2019).

Entrega Contínua

Através da entrega contínua é possível ir um pouco mais além da integração contínua, implantando as mudanças em um ambiente de testes de maneira contínua e preparando para a entrega a cada alteração no código. Desta forma, os desenvolvedores sempre terão um artefato pronto para implantação, que já passou por um processo de testes padronizado (AWS DEVOPS, 2019).

Infraestrutura como Código

A prática de infraestrutura como código determina que as configurações e *scripts* de configuração e instalação devem ser versionados em um repositório assim como o código desenvolvido. Isto permite maior padronização, além de proporcionar que as alterações possam ser auditadas (4LINUX, 2019).

Provisionamento Dinâmico

A infraestrutura como código possibilita que os ambientes possam ser facilmente replicados. É possível utilizar deste conceito também para o provisionamento dinâmico ambientes. Por exemplo, através do uso de *máquinas virtuais* ou de *contêineres* é possível escalar serviços localmente ou em nuvem sempre quando necessário.

Monitoramento

Através do monitoramento contínuo é possível monitorar o desempenho da aplicação e também a experiência do usuário (AWS DEVOPS, 2019). Isto permite que os dados possam ser analisados e gerar *insights*, além de poderem ser utilizados para a detecção de falhas ou gargalos no processo. O monitoramento em tempo real, também pode ser integrado com técnicas como o provisionamento dinâmico, permitindo que as soluções escalem sem afetar a experiência do usuário.

Gestão de Incidentes

Caso algum incidente seja identificado durante o monitoramento, pode ser necessária a realização de algum *roll-back*. Para isto, é necessário que existam estratégias de gestão de incidentes, bem como políticas de *rollback* e *backup* bem definidas (4LINUX, 2019).

Microserviços

A arquitetura de microserviços permite que as aplicações sejam construídas de forma mais flexível. Assim, as aplicações podem ser quebradas em aplicações menores, mais simples e independentes umas das outras. Em contrastes com aplicações monolíticas que são fortemente acopladas, os microserviços são fracamente acoplados, se comunicando apenas com os demais serviços somente quando necessário, através de interfaces bem definidas.

Além disto, os microserviços também facilitam a escalabilidade da solução, otimizando a alocação de recursos. Pois a arquitetura permite que mais recursos sejam alocados aos serviços mais requisitados, sem haver a necessidade de alocar mais recursos para serviços pouco requisitados.

Comunicação e Colaboração

Práticas como comunicação e colaboração são imprescindíveis na cultura *DevOps*. A comunicação deve ocorrer de maneira constante e ser facilitada, permitindo que os profissionais compartilhem informações. Através da colaboração também é possível gerar um senso de responsabilidade compartilhado entre os diversos profissionais.

A comunicação facilitada e colaboração deve ocorrer não somente entre os membros dos times de tecnologia, mas também entre todas as áreas da organização. Tudo isto colabora para que todas as partes da organização estejam alinhadas, e assim permitindo que possam atingir os seus objetivos (AWS DEVOPS, 2019).

1.5 Ciclo DevOps

O ciclo de vida *DevOps* passa por diversos estágios, sendo realimentado continuamente. A Figura 1.2 ilustra as etapas do ciclo *DevOps*, definido nas seguintes etapas: Planejamento (*Plan*), Criação (*Create*), Verificação (*Verify*), Pacote (*Package*), Liberação (*Release*), Configuração (*Configure*) e Monitoramento (*Monitor*).

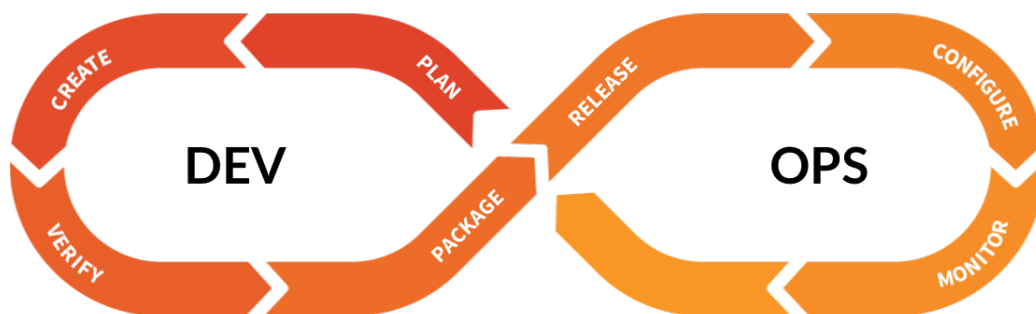


Figura 1.2: Estágios do Ciclo *DevOps*.

Fonte: Adaptada de (GITLAB, 2019).

Todas estas etapas possuem diversas atividades a serem desempenhadas, bem como existem diversas ferramentas que podem ser utilizadas em cada uma delas.

Planejamento – *Plan*

Planejamento é uma atividade recorrente, que tem por objetivo mapear as necessidades do negócio para as aplicações a serem desenvolvidas ou mantidas. Esta etapa normalmente é subdividida em duas partes: *definição* e *planejamento*.

Diversos profissionais podem ser incluídos nesta etapa, dependendo de algumas características da aplicação a ser liberada (tamanho, complexidade, severidade, impacto, entre outras). Por exemplo, desenvolvedores, arquitetos de software, donos da aplicação e responsáveis por infraestrutura, entre outros, podem estar presentes nesta etapa (LITTLE; HERSCHMANN, 2017).

Diversas ferramentas podem auxiliar cada uma delas, ajudando os times a se organizarem, planejarem, se manterem alinhados e poder rastrear o projeto. Desta forma, as equipes podem trabalhar no tempo correto e mantendo a visibilidade e rastreabilidade durante todo o ciclo de vida, da ideia até a produção (GITLAB, 2019). Para a etapa de definição podem ser usadas ferramentas de prototipagem e diagramação, enquanto na etapa de planejamento podem ser usadas ferramentas como quadros de *kanban*, rastreamento de tempo, entre outras ferramentas.

Criação – *Create*

A etapa de criação inclui todas as atividades associadas com a criação do código da versão candidata a lançamento (*Release Candidate*). Esta etapa pode ser subdividida em três partes: Codificação, Compilação e Configuração (LITTLE; HERSCHMANN, 2017).

Para esta etapa, podem ser utilizadas diversas ferramentas. Por exemplo, Ambiente de Desenvolvimento Integrado (*Integrated Development Environment – IDE*) e Sistemas de Controle de Versão (*Source Code Management – SCM*) para a parte de codificação; Ferramentas de automação de compilação, para a parte de compilação; Sistemas de Gerenciamento de Configuração (*Configuration Management System – CMS*), para a parte de configuração.

Verificação – Verify

Esta etapa inclui todas as atividades necessárias para garantir a qualidade da entrega, como por exemplo, testes automatizados, análise estática, testes de segurança, desempenho e aceitação do usuário. No Capítulo 4 detalharemos os tipos de testes existentes e no Capítulo 5 as ferramentas que podem ser utilizadas para cada tipo de teste.

Pacote – Package

Na etapa de pacote são executadas as atividades necessárias para preparação do software para lançamento e implantação. As atividades desta etapa são determinadas pela forma como a aplicação será liberada para o ambiente de produção (LITTLE; HERSCHMANN, 2017). Configurações de pacotes e dependências são exemplo de atividades que figuram nesta etapa.

Liberação – Release

Nesta etapa, são incluídas atividades como agendamento, orquestração, provisionamento e implantação do software no ambiente de produção (LITTLE; HERSCHMANN, 2017). Ferramentas de implantações automatizadas e de gerenciamento de liberação e implantação podem ser utilizadas.

Configuração – Configure

Na etapa de configuração são realizadas as configurações nos ambientes de aplicação que não puderam ser realizada nas etapas anteriores. Por exemplo, configurações de hospedagens compartilhadas, armazenamento, banco de dados e rede (LITTLE; HERSCHMANN, 2017). Nesta etapa podem ser utilizadas ferramentas de gerenciamento de configuração, ferramentas de automação de infraestrutura e ferramentas de provisionamento.

Monitoramento – Monitor

A etapa de monitoramento foca em aspectos como a saúde da infraestrutura e o impacto das entregas. O monitoramento permite que as organizações possam identificar problemas de versões específicas e entender o impacto na experiência do usuário. Além disso, podem ser utilizadas ferramentas de monitoramento durante todo o ciclo *DevOps*, monitorando as atividades e métricas do ciclo. Desta forma, as informações relevantes podem ser utilizadas para realimentar as etapas do próximo ciclo (LITTLE; HERSCHMANN, 2017).

Existem ferramentas de monitoramento para diversos aspectos como desempenho, infraestrutura, aplicação, redes. Além disso, também podem ser utilizadas outras ferramentas como ferramentas de gerenciamento de *logs* e ferramentas de gerenciamento de incidentes. Para uma melhor interpretação dos dados, ferramentas de análise e visualização de dados também podem ser empregadas.

1.6 Conclusões

Neste capítulo, foram introduzidos os principais conceitos e práticas relacionados à *DevOps*. A Seção 1.1 traz uma definição ampla do termo *DevOps*, ressaltando que o *DevOps* trata-se de uma nova forma de trabalho, que tem por objetivo melhorar como as pessoas trabalham em conjunto e não apenas de automação de trabalhos manuais ou da adoção de práticas isoladas. Em seguida a

Seção 1.2 conta um pouco da história do surgimento do movimento *DevOps* e quais os movimentos que o inspiraram. Na Seção 1.3 apresentamos a estrutura **CALMS**, que são pilares da filosofia do *DevOps* e também alguns benefícios da adoção do *DevOps*. Diversas práticas do *DevOps* são apresentadas na Seção 1.4. Por fim, a Seção 1.5 apresenta a os estágios do ciclo *DevOps*, quais atividades compõem cada uma de suas etapas e quais tipos de ferramentas podem ser utilizadas em cada uma delas.

A black and white photograph showing two hands holding two interlocking puzzle pieces. The hands are positioned on either side of the pieces, with fingers gripping them. The puzzle pieces are light-colored and have a standard interlocking shape. The background is a plain, light color.

2. Integração Contínua

O objetivo deste capítulo é descrever melhor quais são os componentes da Integração Contínua e como eles funcionam. Além disto, também apresentamos uma série de práticas para tornar a Integração Contínua mais efetiva. Espera-se que ao término deste capítulo, o estudante consiga responder à algumas perguntas como:

- O que é Integração Contínua?
- Como tornar a IC mais efetiva?
- Quais são os componentes da IC?
- Como funciona um Sistema de Controle de Versão?
- Como funciona um Sistema de Automação de build?
- Quais são as principais funcionalidades de um Sistema de IC?

2.1 Integração Contínua

A Integração Contínua – IC (*Continuous Integration – CI*) é uma prática de desenvolvimento de software onde os membros de um time integram seus trabalhos frequentemente, usualmente cada pessoa integra o seu trabalho no mínimo uma vez ao dia, levando a múltiplas integrações por dia (FOWLER, MARTIN, 2006). Desta forma, após cada alteração ser integrada, ela deve ser verificada através de um *build* automático e testes para detectar erros o mais rápido possível. Desta forma, a integração deixa de ser um evento ou marco do projeto (DUVALL; MATYAS; GLOVER, 2007).

A integração contínua surgiu como uma boa prática visto que os desenvolvedores de software frequentemente trabalham de forma isolada necessitando integrar as suas alterações com a base de código do time. Esperar dias ou mesmo semanas para realizar a integração pode ocasionar diversos conflitos durante a integração, falhas difíceis de serem detectadas e corrigidas, estratégias de código divergentes e duplicação de esforços (GUCKENHEIMER, SAM, 2017).

Para a correta implementação da IC são necessários alguns elementos fundamentais:

- Sistema de controle de versão;

- Processo de *build* automático ¹;
- Sistema de Integração Contínua.

Para cada um deles, o time deve escolher as ferramentas e as abordagens que irão utilizar.

Os sistemas de controle de versão são uma parte fundamental em projetos de desenvolvimento. O repositório deve conter não somente o código da aplicação, mas também todos os artefatos necessários para realizar a compilação da aplicação. Como por exemplo *scripts* de testes, arquivos de configuração, esquemas de bancos de dados, *scripts* de instalação e bibliotecas de terceiros (FOWLER, MARTIN, 2006).

Através de um sistema de controle de versão, os times podem criar ramificações (*branches*) de curto prazo para isolar o desenvolvimento de novas funcionalidades. Assim que uma funcionalidade é concluída, o desenvolvedor pode fundi-la à ramificação principal. Desta forma, os desenvolvedores devem definir um modelo de ramificação (*branch model*) que melhor se adapte à sua realidade, bem como garantir que todo *commit* realizado na ramificação principal irá disparar os processos de *build* e testes automatizados (GUCKENHEIMER, SAM, 2017).

A automação do processo de compilação e construção dos artefatos, irá garantir que o processo seja feito de maneira padronizada e que as dependências corretas sejam utilizadas. Implementar o processo de IC conforme descrito irá garantir que os defeitos sejam detectados antes no ciclo de desenvolvimento, tornando sua correção menos custosa. Além disto, os testes automatizados irão garantir a consistência em termos de qualidade (GUCKENHEIMER, SAM, 2017).

As funcionalidades principais de um sistema de IC são bem simples: compilar o software, executar os testes e relatar os resultados. Os sistemas de integração contínua compilam e testam automaticamente e periodicamente o software. O principal benefício é evitar longos períodos entre as compilações e testes. Estes sistemas também podem simplificar e automatizar a execução de muitas tarefas, como por exemplo, execução testes multi-plataforma ou testes de desempenho (C. T. BROWN; CANINO-KONING, 2011).

2.1.1 Componentes da Integração Contínua

A Figura 2.1 traz os componentes de um cenário típico de integração contínua. Em Duvall, Matyas e Glover (2007), temos uma descrição deste cenário nas seguintes etapas:

1. O desenvolvedor efetua o *commit* do código para o repositório do sistema de controle de versão. Enquanto isto, o Sistema de IC monitora este repositório de tempos em tempos;
2. Assim que um *commit* ocorre, o Sistema de IC detecta a alteração e obtém a versão mais recente e executa um *script* que irá realizar o *build* e disponibilizar para a execução de testes, integração com o banco de dados, entre outras atividades.
3. Ao término da execução do *script* o servidor de IC deverá reportar os resultados de alguma forma, por exemplo enviando um *e-mail* para as partes interessadas ou exibindo os resultados em alguma interface.
4. O processo é repetido e o servidor de IC continua constantemente monitorando por alterações.

2.2 Práticas da Integração Contínua

Fowler, Martin (2006) apresenta uma série de práticas para tornar a Integração Contínua mais efetiva, que aqui resumimos da seguinte forma:

¹O termo *build* aqui utilizado não consiste apenas da compilação do código (ou variações dependendo da linguagem), mas sim como um processo de reunir o código-fonte e verificar se o software funciona de maneira coesa. O processo pode incluir a compilação, teste, inspeção, implantação, entre outras atividades.

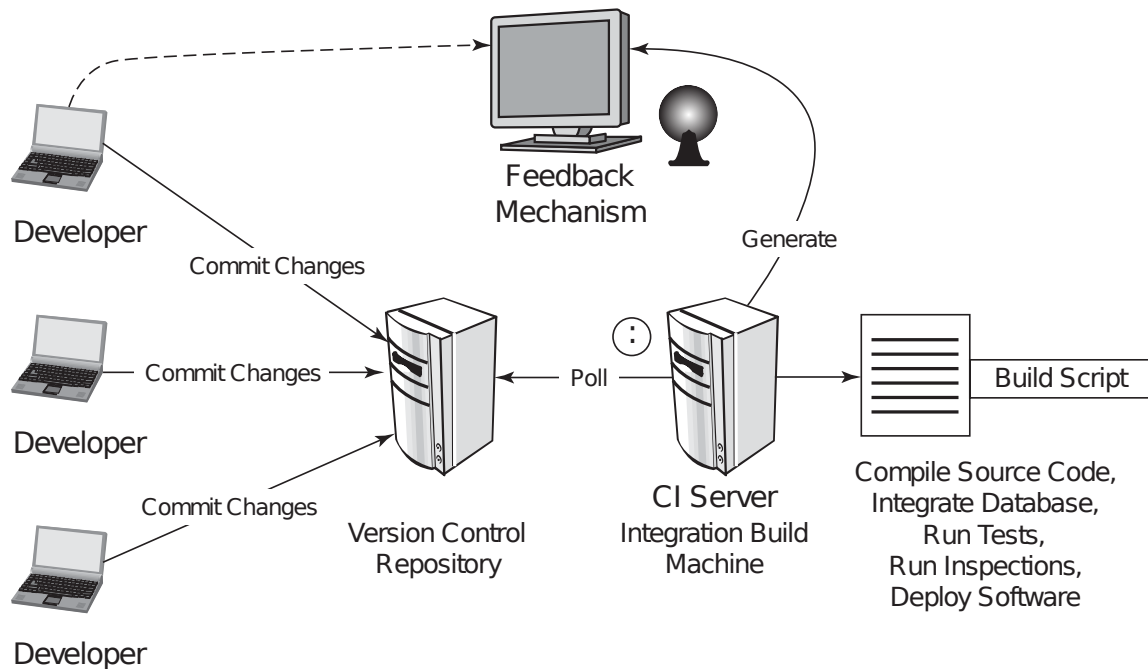


Figura 2.1: Os componentes da Integração Contínua.
Fonte: Adaptada de (DUVALL; MATYAS; GLOVER, 2007).

- **Manter um único repositório de código:** A construção de um projeto software pode necessitar diversos arquivos, sejam códigos de aplicação e de testes, arquivos de configurações, *scripts* de instalação, esquemas de banco de dados, bibliotecas de terceiros, etc. O ideal é que estes arquivos estejam todos no repositório, de forma que o que for necessário para realizar o *build* e executar o software esteja contido no repositório.
- **Automatizar o processo de *build*:** Tornar os códigos fontes em um software executando pode ser um processo complicado que envolve diversas atividades como compilação, movimentação de arquivos, carregar esquemas nas bases de dados, entre outras coisas. Complementando a regra anterior, após obter os fontes do repositório deve ser possível ter o software executando na máquina através de um comando.
- **Torne seu software auto-testável:** Testes podem não serem perfeitos, mas mesmo assim podem auxiliar a detectar diversas falhas no software, o suficiente para justificar sua utilização. Para que o software seja auto-testável são necessários testes automatizados que possam checar a grande maioria do seu código. Além disso, os testes devem poder ser executados de maneira simples, através de um comando.
- **Integrar as atualizações todos os dias:** A integração trata-se primordialmente sobre comunicação, permitindo que os desenvolvedores comuniquem sobre as alterações realizadas, permitindo que as demais pessoas tenham visibilidade assim que a alteração ocorreu. O único pré-requisito para isto é que antes do desenvolvedor realizar o *commit* ele deve conseguir executar corretamente o software incluindo os testes.
- **Cada *commit* deve atualizar o repositório principal em uma máquina de integração:** Realizando *commits* diários, o time irá frequentemente testar o software, isto significa que a base de código sempre se manterá em um estado saudável. No entanto, na prática as coisas

podem dar um pouco errado, seja por diferenças entre os ambientes, ou mesmo pela falta de disciplina de um desenvolvedor que não testou antes de submeter o *commit*. O desenvolvedor só deve considerar o *commit* como feito após o *build* ter sido bem sucedido em uma máquina de integração. Assim o desenvolvedor que realizou o *commit* é responsável por monitorar o processo e deverá realizar as alterações caso ele quebre.

- **Corrija *builds* quebrados imediatamente:** Uma parte fundamental de realizar *builds* constantemente é que caso alguma coisa quebre ela deve ser consertada rapidamente. O ponto fundamental da IC é o desenvolvedor saber que possui uma base de código estável. Não é algo muito problemático o *build* eventualmente quebrar, porém caso isso aconteça com frequência, pode significar que as pessoas não estão tomando o cuidado de realizar os testes localmente antes de realizar os *commits*.
- **Mantenha o processo de *build* rápido:** O ponto fundamental da IC é proporcionar *feedback* rápido. Caso atividades como realizar o processo de *build* demorem um longo período de tempo. Determinados casos podem exigir testes mais demorados, caso isto ocorra, o ideal é começar a trabalhar com um *pipeline*. Assim o *commit* irá disparar o primeiro processo de *build* que irá executar rapidamente, contendo alguns casos de testes mais básicos. Após esse *build* ser completado, testes adicionais mais complexos e demorados podem ser iniciados. O ponto principal é encontrar um balanceamento em que o processo de encontrar falhas mais rapidamente e obter um *build* estável o suficiente para que as pessoas possam trabalhar.
- **Teste em uma cópia do ambiente de produção:** O objetivo da realização de testes é reproduzir sob condições controladas, qualquer problema que os softwares em produção podem vir a apresentar. Ao executar testes em um ambiente diferente, qualquer diferença resulta em um risco de que o que acontece durante os testes podem não acontecer no ambiente de produção. O ideal é que o ambiente de testes seja uma cópia o mais exata possível do ambiente de produção, utilizando o mesmo sistema operacional, sistema de banco de dados, etc. Em alguns casos, por exemplo, em desenvolvimento de sistemas para *desktop* ou aplicativos móveis é impraticável testar em todas as plataformas disponíveis. Apesar disto, utilizando máquinas virtuais, contêineres e/ou emuladores é possível simular em configurações distintas reduzindo esta lacuna entre o ambiente de desenvolvimento e de produção.
- **Torne fácil para obter os últimos executáveis:** Qualquer um envolvido com o projeto de software deve poder obter os últimos executáveis e conseguir executá-los: para demonstrações, testes exploratórios ou apenas para verificar as últimas alterações. Garantindo que qualquer pessoa envolvida saiba onde estão os últimos executáveis torna-se mais fácil para que alguém possa identificar algo que não esteja de acordo e dizer o que deve ser alterado.
- **Todos podem ver o que está acontecendo:** Integração Contínua é primordialmente sobre comunicação, então devemos garantir que todos possam facilmente ver o estado do software e as mudanças que foram feitas sobre ele. Uma das coisas mais importantes é o estado do *build* principal. Em um sistema de IC usualmente existe uma interface onde é possível acompanhar se existe algum *build* em progresso, bem como o estado dos últimos *builds* realizados.
- **Automatize a implantação do sistema:** Para fazer IC são necessários múltiplos ambientes, um para realizar os testes de *commit*, um ou mais para executar testes secundários. Como isto pode implicar em mover executáveis entre estes ambientes diversas vezes por dia, é importante ter *scripts* que irão permitir implantar a aplicação facilmente em qualquer ambiente. Uma consequência natural é que podemos adicionalmente criar *scripts* que permitam a implantação em produção de maneira similar. Caso a implantação seja feita em produção, uma capacidade adicional a ser considerada é a habilidade de realização de *rollback* automaticamente caso

alguma coisa dê errado.

2.3 Sistemas de Controle de Versão

Um sistema de controle de versão é um sistema que registra alterações sobre um arquivo ou um conjunto de arquivos ao longo do tempo, de forma que seja possível retornar à versões específicas posteriormente (CHACON; STRAUB, 2014).

Um sistema de controle de versão permite que reverter arquivos específicos ou mesmo um projeto inteiro para um estágio anterior. Também permite que as alterações ao longo do tempo possam ser comparadas, verificar quem realizou alterações, e caso ocorra algum problema facilmente analisar o que foi alterado, entre outras coisas. Utilizar um sistema de controle de versões geralmente significa que caso alguma coisa seja estragada ou perdida, podemos recuperá-la facilmente (CHACON; STRAUB, 2014).

Uma forma simples e primitiva de se realizar controle de versão pode ser manter diversas cópias dos arquivos manualmente. A Figura 2.2 traz uma tirinha ilustrando os problemas que essa forma manual de versionamento pode trazer. Além disto, esta forma pode ser totalmente passível de erros, pois pode-se acidentalmente sobrescrever algum arquivo de forma indevida.

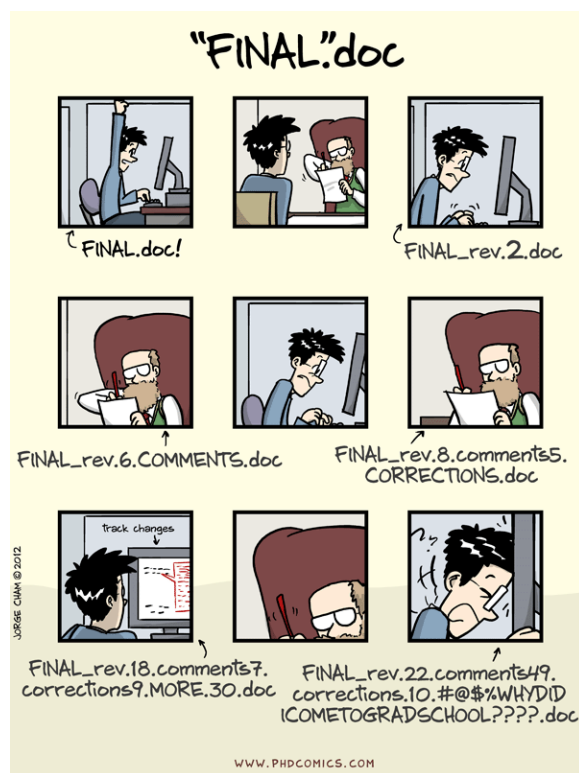


Figura 2.2: Tirinha Ilustrando uma Forma Manual de Versionamento.

Fonte: <http://phdcomics.com/comics/archive.php?comid=1531>.

Para lidar com estes problemas, os Sistemas de Controle de Versão (*Version Control Systems* – VCS) foram criados. Podemos dividir os sistemas de controle de versão em três categorias: locais, centralizados e distribuídos. A Figura 2.3 traz um comparativo da forma de funcionamento de cada uma destas categorias.

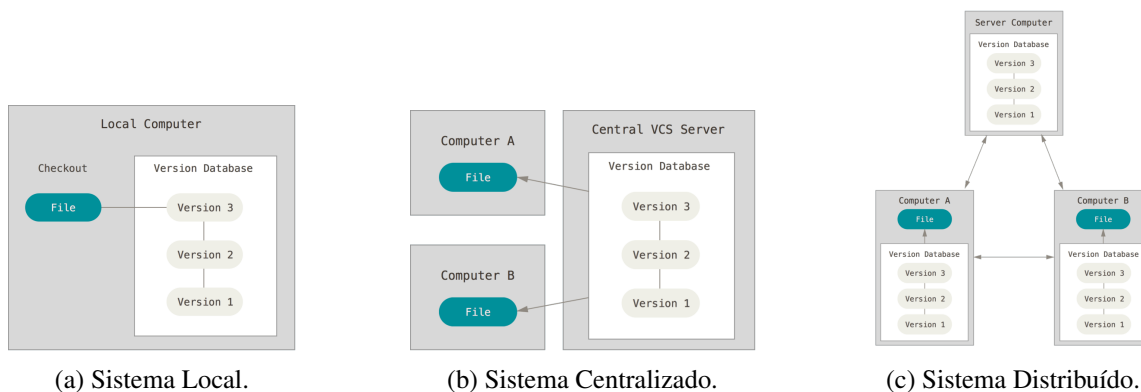


Figura 2.3: Diferentes Categorias de Sistemas de Controle de Versão.

Fonte: (CHACON; STRAUB, 2014).

2.3.1 Sistemas Locais de Controle de Versão

Para lidar com este problema, há bastante tempo os programadores desenvolveram VCS locais. Uma das ferramentas mais populares desta categoria é o *Revision Control System* (RCS), desenvolvido em 1982 e que ainda hoje é distribuído juntamente com diversos sistemas operacionais. O RCS funciona mantendo as alterações entre os arquivos em um formato especial no disco, para que ele possa recriar um arquivo tal como ele era em um determinado ponto no tempo (CHACON; STRAUB, 2014).

2.3.2 Sistemas Centralizados de Controle de Versão

Os desenvolvedores usualmente precisam colaborar com outros desenvolvedores, então para lidar com isto os Sistemas Centralizados de Controle de Versão foram criados. Sistemas como o *Subversion*, *CVS* e *Perforce*, trabalham com um servidor central que contém todos os arquivos de controle de versão (CHACON; STRAUB, 2014).

Este tipo de versionamento oferece algumas vantagens, especialmente sobre os VCS Locais, trazendo maior visibilidade. Além disso, os administradores podem ter um maior controle sobre as ações que cada um pode realizar. A maior desvantagem deste tipo de VCS é que ele possui um único ponto de falha, se ocorrer um problema no servidor, as pessoas podem não conseguir salvar as alterações ou mesmo perder todo o trabalho caso não existam *backups* em caso de uma falha mais grave (CHACON; STRAUB, 2014).

2.3.3 Sistemas Distribuídos de Controle de Versão

Diferente dos VCS Centralizados, os VCS Distribuídos duplicam localmente o repositório completo. Assim caso ocorra algum problema no servidor, qualquer um dos repositórios de clientes pode ser utilizado para restaurá-lo (CHACON; STRAUB, 2014).

Este tipo de sistema, permite que se trabalhe com diversos repositórios remotos, permitindo que se possa configurar diversos fluxos de trabalho que não são possíveis em sistemas centralizados com modelos hierárquicos (CHACON; STRAUB, 2014). Como exemplos deste tipo de sistema temos o *Git*, *Mercurial* e *Bazaar*.

2.4 Automação de *Build*

O *build* é uma das funcionalidades mais básicas de um sistema de IC, por vezes tornando-se um sinônimo de IC. Escrevendo *scripts* de *build* automatizados, reduzimos o número de atividades manuais, repetitivas e passíveis de erro executadas em um projeto de software (DUVALL; MATYAS; GLOVER, 2007).

Além disso, é importante desacoplar os *scripts* de *build* do Ambiente de Desenvolvimento Integrado (*Integrated Development Environment – IDE*). Em outras palavras, uma IDE pode depender de um *script* de *build*, porém um *script* de *build* não deve depender de uma IDE. A criação de *scripts* separados é importante por algumas razões. Por exemplo, a IDE pode incluir dependências dentro do diretório de instalação, os arquivos podem ser em um formato proprietário, desenvolvedores utilizando ferramentas diferentes, etc. Além disto, para executar o *build* em um servidor de IC, é necessário que o *build* possa ser executado automaticamente sem que haja alguma intervenção (DUVALL; MATYAS; GLOVER, 2007).

Uma das ferramentas mais difundidas no âmbito de automação de *build* é o *Make*, especialmente em programas escritos em linguagem C. O *Make* é uma ferramenta de automação de *build* para geração de executáveis e bibliotecas a partir do código-fonte. O *Make* opera utilizando arquivos *Makefiles*, que especificam como derivar o programa alvo a partir de suas dependências que direcionam o *Make* sobre como realizar a compilação e vinculação (*linking*) do programa. Existem diversas implementações diferentes da ferramenta como BSD², GNU³, *nmake*⁴.

Para o ecossistema da (*Java Virtual Machine – JVM*), as convenções utilizadas na linguagem C não atendem perfeitamente as necessidades, assim novas ferramentas foram criadas como alternativas ao *Make*. Atualmente, as ferramentas mais utilizadas para a realização desta tarefa são: *Apache Ant*, *Apache Maven* e *Gradle* (BAELDUNG, 2018).

2.4.1 *Make*

O Código Fonte 2.1 traz um exemplo de *Makefile* para um programa Java. A regra de como obter os arquivos *.class* a partir dos arquivos *.java* é definida pela entrada `.java.class:`. A entrada segue o formato: sufixo dependência + sufixo do alvo, logo em seguida vem a regra para esta definição que utiliza as variáveis definidas anteriormente e a macro pré-definida `$*`, que irá retornar o nome base para o alvo utilizado. `CLASSES = ...`, é uma macro definida contendo o nome de cada um dos arquivos fonte java. A entrada `classes: $(CLASSES:.java=.class)`, realiza a substituição do sufixo *.java* pelo sufixo *.class* em todas as entradas da macro `CLASSES`. Por fim, o alvo `clean:`, define um novo alvo, que irá remover todos os arquivos *.class* através da macro pré-definida `$(RM)` (NEWHALL, TIA, 2019)

Neste exemplo, o *Make* apenas irá realizar a geração dos arquivos *.class*, através do comando `make classes` ou apenas `make`, que irá executar o primeiro alvo do arquivo (neste caso `default: classes`).

Caso seja necessário, o comando `make clean` pode ser utilizado para limpar os arquivos *.class* antes da execução de um novo *build*. A ferramenta realiza o *build* de um arquivo alvo apenas se ele não existir ou se alguma de suas dependências tenham sido modificadas desde o último *build*.

²[https://www.freebsd.org/cgi/man.cgi?make\(1\)](https://www.freebsd.org/cgi/man.cgi?make(1))

³<https://www.gnu.org/software/make/>

⁴<https://docs.microsoft.com/en-us/cpp/build/reference/nmake-reference>

```
1  # Especifica o compilador
2  JC = javac
3
4  # Especifica as opções do compilador
5  JFLAGS = -g
6
7  # Necessário para reconhecer .java e .class como extensões (sufixos)
8  .SUFFIXES: .java .class
9
10 # Regra de compilação de arquivos
11 .java.class:
12     $(JC) $(JFLAGS) $*.java
13
14 # Lista dos arquivos fontes
15 CLASSES = \
16     Foo.java \
17     Blah.java \
18     Library.java \
19     Main.java
20
21 # Alvo padrão do Make
22 default: classes
23
24 classes: $(CLASSES:.java=.class)
25
26 clean:
27     $(RM) *.class
```

Código Fonte 2.1: Exemplo de arquivo *Makefile*.

Fonte: Adaptado de (NEWHALL, TIA, 2019).

2.4.2 Apache Ant

O *Apache Ant* ⁵ é uma biblioteca Java utilizada para automação do processo de *build* de aplicações Java, mas também pode ser utilizada para outras aplicações. Foi inicialmente construída como parte do projeto *Apache Tomcat* ⁶ e depois se tornou um projeto separado (BAELDUNG, 2018).

O *Ant* é similar ao *Make* em diversos aspectos, também definindo os processos do *build* como alvos. Os arquivos de *build* do *Ant* são escritos em XML e por convenção são chamados *build.xml* (BAELDUNG, 2018).

O principal benefício do *Ant* é sua flexibilidade, pois não impõe convenções de código ou de estrutura de projetos. Consequentemente, isto requer que os desenvolvedores escrevam os comandos, o que pode resultar em arquivos XML gigantescos e difíceis de manter (BAELDUNG, 2018).

Inicialmente, o *Ant* não tinha nenhum suporte nativo para gerenciamento de dependências.

⁵<https://ant.apache.org/>

⁶<https://tomcat.apache.org/>

Alguns anos depois o *Apache Ivy*⁷ foi desenvolvido como um subprojeto e foi integrado ao *Ant* (BAELDUNG, 2018).

O Código Fonte 2.2, traz um exemplo de arquivo *build.xml* que define 4 alvos: *init*, *compile*, *dist* e *clean*. Neste exemplo, a compilação pode ser realizada através do comando `ant compile`, que irá invocar o alvo *compile*. Como existe uma dependência do alvo *compile* para o alvo *init*, o *Ant* irá resolver esta dependência e também irá disparar o alvo *init*. Para produzir os arquivos *jar* para distribuição, podemos utilizar o comando `ant dist`, que por sua vez também depende do alvo *compile*. Podemos remover os diretórios definidos pelas propriedades `${build}` e `${dist}` utilizando o comando `ant clean`.

```
1 <project name="MyProject" default="dist" basedir=".">
2   <description>
3     exemplo de arquivo de build
4   </description>
5   <!-- define propriedades globais -->
6   <property name="src" location="src"/>
7   <property name="build" location="build"/>
8   <property name="dist" location="dist"/>
9
10  <target name="init">
11    <!-- Cria as propriedades DSTAMP, TSTAMP, e TODAY -->
12    <tstamp/>
13    <!-- Cria a estrutura de diretório de build
14    utilizada pelo compilador -->
15    <mkdir dir="${build}"/>
16  </target>
17
18  <target name="compile" depends="init"
19    description="compila os fontes">
20    <!-- Compila o código Java de ${src} para ${build} -->
21    <javac srcdir="${src}" destdir="${build}"/>
22  </target>
23
24  <target name="dist" depends="compile"
25    description="gera a distribuição">
26    <!-- Cria o diretório de distribuição -->
27    <mkdir dir="${dist}/lib"/>
28
29    <!-- Copia os arquivos em ${build} para o arquivo
30    MyProject-${DSTAMP}.jar -->
31    <jar jarfile="${dist}/lib/MyProject-${DSTAMP}.jar"
32      basedir="${build}"/>
33  </target>
34
```

⁷<https://ant.apache.org/ivy/>

```
35 <target name="clean" description="clean up">
36   <!-- Apaga os diretórios ${build} e ${dist} -->
37   <delete dir="${build}"/>
38   <delete dir="${dist}"/>
39 </target>
40 </project>
```

Código Fonte 2.2: Exemplo de arquivo *build.xml*.

Fonte: Adaptado de (APACHE SOFTWARE FOUNDATION, 2019a).

2.4.3 Apache Maven

*Apache Maven*⁸ é uma ferramenta para gerenciamento de dependências e automação de *build*, usada principalmente para aplicações Java. Assim como o *Ant*, o *Maven* também utiliza arquivos XML. Enquanto o *Ant* garante flexibilidade, mas requer que tudo seja escrito do zero, o *Maven* conta com convenções e comando pré-definidos. O arquivo de configuração do *Maven*, por convenção chamado *pom.xml*, contém tanto as instruções para a realização do *build*, como para gerenciar as dependências (BAELDUNG, 2018).

O *Maven* espera que os arquivos estejam em uma estrutura de diretórios específicas. A Tabela 2.1 traz esta estrutura de diretórios esperados pelo *Maven*. Esta estrutura permite que usuários de um projeto *Maven*, rapidamente possam se familiarizar com outros projetos. Apesar da conformidade com esta estrutura ser encorajada, também é possível sobrescrever as configurações via o descritor de projeto (APACHE SOFTWARE FOUNDATION, 2019b).

O arquivo *Project Object Model* – *POM* é o arquivo descritor do projeto, por convenção é chamado de *pom.xml*. Este arquivo é o núcleo da configuração de um projeto no *Maven*, sendo um único arquivo que contém a maior parte da informação necessária para realizar o *build* de um projeto da maneira desejada (APACHE SOFTWARE FOUNDATION, 2019b).

O Código Fonte 2.3 traz um exemplo deste arquivo. Neste arquivo, temos diversas chaves, vale destacar algumas delas. A chave `modelVersion` indica qual versão o arquivo POM está utilizando. A chave `groupId` indica qual é o identificador único da organização ou grupo que criou o projeto. A chave `artifactId` diz qual é o nome base do artefato principal sendo gerado pelo projeto. Um artefato produzido pelo *Maven*, tipicamente receberá o nome seguindo este padrão: `<artifactId>-<version>.<extension>`.

```
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
4     http://maven.apache.org/xsd/maven-4.0.0.xsd">
5   <modelVersion>4.0.0</modelVersion>
6   <groupId>com.mycompany.app</groupId>
7   <artifactId>my-app</artifactId>
8   <version>1.0-SNAPSHOT</version>
9
```

⁸<https://maven.apache.org/>

Tabela 2.1: Estrutura de Projetos Padrão do *Maven*.

Fonte: Adaptada de (APACHE SOFTWARE FOUNDATION, 2019b).

Localização	Diretório/Arquivo	Descrição
src/main/java	Diretório	Arquivos fontes da Aplicação/Biblioteca.
src/main/resources	Diretório	Recursos da Aplicação/Biblioteca (arquivos de configuração e outros arquivos).
src/main/filters	Diretório	Arquivos que podem ser configurados conforme o ambiente e/ou <i>profile</i> .
src/main/webapp	Diretório	Arquivos fontes da aplicação Web.
src/test/java	Diretório	Análogos aos itens anteriores porém relacionados aos testes.
src/test/resources	Diretório	
src/test/filters	Diretório	
src/it	Diretório	Testes de Integração (utilizado principalmente <i>plugins</i>).
src/assembly	Diretório	Descritores de configuração da construção de artefatos.
src/site	Diretório	Configuração do site de documentação do projeto.
target	Diretório	Arquivos de saída do <i>build</i> .
pom.xml	Arquivo	Descritor do projeto.
LICENSE.txt	Arquivo	Licença do projeto.
NOTICE.txt	Arquivo	Contém avisos e atribuições necessárias para uso de bibliotecas as quais o projeto depende.
README.txt	Arquivo	Leia-me do projeto.

```

10  <properties>
11    <maven.compiler.source>1.7</maven.compiler.source>
12    <maven.compiler.target>1.7</maven.compiler.target>
13  </properties>
14
15  <dependencies>
16    <dependency>
17      <groupId>junit</groupId>
18      <artifactId>junit</artifactId>
19      <version>4.12</version>
20      <scope>test</scope>
21    </dependency>
22  </dependencies>
23 </project>

```

Código Fonte 2.3: Exemplo de arquivo *pom.xml*.

Fonte: Adaptado de (APACHE SOFTWARE FOUNDATION, 2019c).

Uma forma simples para invocar o *Maven*, é utilizando o comando `mvn <fase>`, onde *fase* é uma fase do ciclo de vida do padrão do *Maven* (APACHE SOFTWARE FOUNDATION, 2019b). O ciclo de vida padrão é definido com as seguintes fases:

- **validate**: Valida se a estrutura do projeto está correta e contém toda informação necessária.
- **compile**: Compila o código fonte do projeto.
- **test**: Testa se os códigos fontes compilados utilizando um *framework* de testes unitários disponíveis.
- **package**: Empacota o código compilado em um formato distribuível, por exemplo, JAR ou WAR.
- **verify**: Executa verificação do pacote, validando se atende os critérios de qualidade.
- **install**: Instala o pacote no repositório local, para usar como uma dependência em outros projetos locais.
- **deploy**: Copia o pacote final para um repositório remoto.

Ao executar uma fase, o *Maven* irá executar as fases anteriores em ordem, por exemplo, ao executar o comando `mvn test` as fases *validate*, *compile*, serão executadas primeiro. Além das fases padrão do ciclo de vida, existem também outras fases:

- **clean**: Limpa artefatos gerados por *builds* anteriores.
- **site**: Gera o site de documentação para o projeto.

Além disto, o *Maven* também possui uma série de *plugins* que podem ser adicionalmente configurados.

2.4.4 Gradle

Apesar de o *Maven* possuir a vantagem de manter o processo de *build* mais simples e padronizado, isto o torna menos flexível que o *Ant*, isto levou à criação do *Gradle* ⁹, que busca combinar o melhor dos dois: as funcionalidades do *Maven* com a flexibilidade do *Ant* (BAELDUNG, 2018).

O *Gradle* é uma ferramenta de gerenciamento de dependências e automação de *build* que foi construída sobre os conceitos do *Ant* e do *Maven*. Porém, ao invés de utilizar arquivos XML como estas ferramentas, ele utiliza uma Linguagem de domínio específico (*Domain-Specific Language – DSL*) baseada na linguagem *Groovy*. Desta forma, os arquivos de configuração ficam mais concisos, uma vez que a linguagem foi desenvolvida especialmente para isto.

Por convenção, o arquivo de configuração do *Gradle* é chamado de *build.gradle*. O Código Fonte 2.4 traz um exemplo deste arquivo. Este exemplo inclui o *plugin java-library*. Em seguida temos `jcenter()`, que define o repositório central de artefatos. Temos diferentes formas de adicionar dependências, por exemplo `api 'org.apache.commons:commons-math3:3.6.1'`, adiciona uma dependência que seja exportada aos consumidores da biblioteca, enquanto que `implementation 'com.google.guava:guava:26.0-jre'` define uma dependência que será usada apenas internamente. Por fim, temos a inclusão da biblioteca de testes *JUnit*.

As funcionalidades principais do *Gradle* não são muitas, no entanto os *plugins* podem ser bastante úteis. Diferente do *Ant*, que usa o conceito de alvos e do *Maven*, que utiliza o conceito de fases, o *Gradle* trata as etapas do *build* por tarefas (*tasks*) (BAELDUNG, 2018).

Para executar o *Gradle*, basta executar o comando `gradle <task>`, por exemplo, `gradle build` irá executar a *task build*. Uma outra forma de executar o *Gradle* é utilizando um *Wrapper*, que é um *script* que irá invocar uma versão especificada do *Gradle*, fazendo o *download* se necessário. Desta forma, o projeto fica padronizado com uma versão específica do *Gradle*, levando a *builds* mais confiáveis e robustos. Além disto, para atualizar para uma nova versão do *Gradle* ou para os diferentes usuários ou ambientes como IDEs ou servidores de IC, bastaria trocar a definição do

⁹<http://gradle.org/>


```
1 plugins {  
2     id 'java-library'  
3 }  
4  
5 repositories {  
6     jcenter()  
7 }  
8  
9 dependencies {  
10     api 'org.apache.commons:commons-math3:3.6.1'  
11  
12     implementation 'com.google.guava:guava:26.0-jre'  
13  
14     testImplementation 'junit:junit:4.12'  
15 }
```

Código Fonte 2.4: Exemplo de arquivo *build.gradle*.

Fonte: Adaptado de (GRADLE, 2019).

Wrapper. Assim, bastaria o comando `./gradlew build`, para executar a *task build*, e o *Wrapper* se encarrega de baixar a versão correta do *Gradle*.

2.5 Sistemas de Integração Contínua

Quando desenvolvemos software, necessitamos verificar se as novas funcionalidades ou as correções realizadas funcionam conforme o esperado. Uma das funcionalidades básicas de um Sistema de Integração Contínua é testar software. Quando um *commit* é enviado ao repositório de código, é responsabilidade do sistema de IC verificar se este *commit* não irá fazer com que algum teste falhe (DAS, 2016).

Dentre as funcionalidades básicas que um sistema de IC deve possuir, podemos citar: obter a versão mais recente do código a partir do repositório, realizar o *build* do software, executar os testes e apresentar os resultados. Este trabalho poderia ser realizado por um simples *script* com execução agendada para ser executado de tempos em tempos. Porém os sistemas de IC existentes geralmente têm muitas funcionalidades além destas básicas. C. T. Brown e Canino-Koning (2011) traz uma lista de funcionalidades adicionais que os sistemas de CI costumam apresentar:

- **Obter ou Atualizar o código:** Os sistemas de IC precisam rastrear as alterações a partir da última versão e/ou obter uma versão completa dos códigos fontes. Isto normalmente significa uma integração com Sistemas de Controle de Versão. Por exemplo, para grandes projetos, obter uma nova cópia dos códigos fontes pode ser custoso em termos de tempo. Por isto os sistemas de IC geralmente podem tanto obter uma nova cópia completa ou atualizar apenas as alterações em uma cópia já existente.
- **Receitas abstratas de *build*:** Para que a configuração, *build* ou testes sejam realizados são necessárias que algumas “receitas” de como isso deve ser feito. Alguns comandos para isto podem variar em diferentes sistemas operacionais ou arquiteturas. Isto significa que pode ser

necessário que essas receitas sejam específicas (potencialmente introduzindo *bugs*) ou que exista algum nível de abstração.

- **Armazenar *status* de *checkout*, *build* e testes:** Para que seja possível analisar os detalhes das execuções do sistema de IC, pode ser necessário armazenar algumas informações. Por exemplo, podem ser armazenados dados como: *status* do *checkout* do sistema de versionamento (arquivos atualizados, identificador da versão), do *build* (avisos ou erros) ou dos testes (cobertura de código, desempenho, uso de memória). Isto pode ser útil para que questionamentos como “a última versão afetou o desempenho de alguma forma?” ou “a cobertura de código mudou drasticamente desde o último mês?” possam ser respondidos com maior facilidade.
- **Liberação de pacotes:** Os *builds* podem produzir pacotes binários ou outros produtos que necessitam estar disponíveis externamente. Para isto, pode ser necessário que os sistemas de IC transfiram os produtos gerados pelo *build* para um repositório centralizado.
- **Gerenciamento de recursos:** Se alguma etapa do *build* exigir muitos recursos de uma máquina, pode ser desejável executá-la condicionalmente. Por exemplo, um *build* poderia esperar até que não exista outros *builds* ou usuários, ou aguardar até que uma quantidade específica de CPU ou memória esteja disponível.
- **Coordenação de recursos externos:** Testes de integração podem depender de recursos não locais, como bancos de dados ou serviços remotos. Assim, o sistema de IC pode precisar coordenar o acesso a estes recursos entre várias máquinas.
- **Reportar o progresso:** Pode ser importante reportar o progresso do *build* de maneira contínua, de modo que o usuário não necessite esperar até o fim da execução para ver algum resultado. Isto pode ser útil especialmente em longos processos de *build*.

Além disto, um sistema de IC também deve ser resistente a falhas, assim como outros tipos de sistemas. Isto significa que se alguma parte do sistema falhar, ele deve ser capaz de recuperar e continuar a partir deste ponto (A. BROWN; DIBERNARDO, 2016).

O sistema também deve lidar bem com a carga de trabalho, de forma que os resultados estejam disponíveis em um tempo razoável caso os *commits* sejam realizados de forma mais rápida do que os testes possam executar. Podemos realizar, por exemplo, isto distribuindo e paralelizando os esforços de testes (A. BROWN; DIBERNARDO, 2016).

2.6 Conclusões

Neste capítulo descrevemos algumas práticas, bem como os componentes que fazem parte da Integração Contínua. Na Seção 2.1 descrevemos o que é Integração Contínua, e introduzimos os elementos fundamentais para que ela possa acontecer. Em seguida, na Seção 2.2, descrevemos uma série de práticas, que se observadas podem tornar a Integração Contínua mais efetiva. Na Seção 2.3 explanamos sobre como funcionam os diferentes tipos de sistema de controle de versão existentes. Depois na Seção 2.4 explicamos sobre a importância da automação do *build* e também introduzimos algumas ferramentas com este objetivo. Por fim na Seção 2.5 apresentamos as funcionalidades básicas e as principais funcionalidades desejáveis em um sistema de IC.

A close-up photograph of several wooden blocks with letters and numbers on them, arranged on a wooden surface. The blocks are spelling out the word "ERROR" in a row. Other blocks with letters like 'A', 'B', 'C', 'S', 'E', 'R', 'O', 'R', 'N', 'W' are scattered around. A semi-transparent blue banner with white text is overlaid on the bottom right of the image.

3. *DevOps*: Modelos de Maturidade, Estatísticos

Neste capítulo pretende-se ampliar a visão sobre *DevOps* abordando os modelos de maturidade, abordar uma visão geral de maturidade no mercado através de estatísticas e fornecer um caminho inicial para estudo de ferramentas envolvidas na metodologia. Ao término, é esperado que o estudante atinja os objetivos abaixo:

- Compreender os diferentes estágios de maturidade na metodologia *DevOps*.
- Compreender o papel dos testes automatizados no modelo de maturidade.
- Compreender o papel da segurança no modelo de maturidade.
- Ter uma visão básica das ferramentas *DevOps* e suas categorias.
- Avaliar o estado da metodologia *DevOps* através de estatísticas.

3.1 Modelos de Maturidade

Como vimos, *DevOps* é uma metodologia de desenvolvimento de software que visa a aproximar as partes envolvidas neste processo, facilitando a comunicação e gerando sinergia que se concretiza em um produto de melhor qualidade. Seu foco não é puramente técnico. Está também relacionado à um conjunto de boas práticas e habilidades comportamentais. Surgiu de um problema que os departamentos de TI enfrentaram ao longo dos anos: equipes especialistas estanques que não se comunicam. Na Figura 3.1, (TILLEY; ROSENBLATT, 2017) mostram o que eles definem como uma típica estrutura de TI.

No mercado de trabalho, é bastante comum observar equipes de TI separadas dessa maneira. A necessidade dessa divisão é bem justificada, pois é uma forma de dividir responsabilidades para poder atender aos demais setores de uma empresa. Inclusive, ainda na figura 3.1, o suporte de sistemas está separado do desenvolvimento, ou seja, quem dá suporte, não são os desenvolvedores, o que pode aumentar a quantidade de entregas. Contudo, vale a pena questionar: quais as consequências da separação entre desenvolvimento, banco de dados, segurança e qualidade? Essa separação, que também acaba sendo física, acaba tendo impacto na comunicação entre essas equipes.

Agora imagine um cenário em que uma equipe possua profissionais de especialidades distintas: desenvolvedores, administradores de banco de dados, analistas de testes e analistas de segurança. Essa

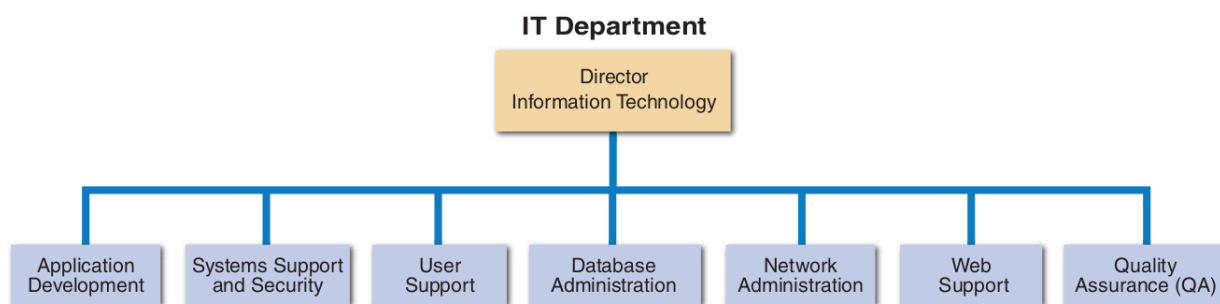


Figura 3.1: Típica Estrutura de TI.

Fonte: Adaptada de (TILLEY; ROSENBLATT, 2017).

é a proposta *DevOps* : criar equipes únicas onde o desenvolvimento e as operações cooperam entre si, visando a entregar um produto de melhor qualidade, apoiando-se em uma melhor comunicação.

Assim como a metodologia Ágil surgiu como alternativa ao modelo cascata de desenvolvimento, o *DevOps* surgiu como alternativa ao modelo compartimentalizado de projetos de software. Comparando as figuras 3.1 e 1.1, que apresentam modelos diferentes de equipes em um setor de TI, pode-se verificar que há uma intencionalidade na metodologia *DevOps* de causar uma ruptura e inovar a forma de se produzir software.

Entre tantas boas práticas e recomendações contidas na metodologia *DevOps* , é fácil que uma equipe se perca ao realizar uma avaliação de seu estágio atual de maturidade. Podem ocorrer enganos, pensando que a direção que está sendo seguida é adequada. Essa situação subjetiva, também dificulta que se saiba como melhorar ainda mais, qual o próximo passo em uma escala de evolução e melhoria contínua. Isso pode levar a um certo comodismo com o estágio atual. Além disso, poder-se comparar com diferentes níveis de qualidade, seria muito bom para as organizações.

Nesse sentido, são apresentados abaixo dois modelos de maturidade: de *DevOps* e de Entrega contínua. Ambos estão estruturados em uma escala de evolução, de Básico à Expert. Estão organizados em tabelas contendo práticas de *DevOps* recomendadas para cada fundamento da metodologia, representados em cada linha. Os modelos propostos são uma estruturação dos conceitos de *DevOps* apresentados no primeiro capítulo, que lhe permitem avaliar a situação atual de uma empresa em sua batalha pela melhoria contínua. É indicado que, durante sua leitura, sejam aplicados os conceitos de *DevOps* , como Filosofia, Práticas e Ciclos, para perceber a intencionalidade de cada recomendação escolhida pelos autores para o modelo.

3.1.1 Modelo de Maturidade *DevOps* - Atlassian

Este modelo, proposto por (ATLASSIAN, INC, 2019), é composto por dez perguntas, com cinco alternativas cada e uma pontuação de 1 a 5 para cada resposta. O modelo foi adaptado na tabela 3.1 para facilitar a compreensão e visualização mais rápida do cenário atual em que a organização se encontra. Na primeira coluna, as perguntas foram alteradas para substantivos que remetem mais rapidamente ao que está sendo avaliado.

O nível básico, neste modelo, apresenta o nível de uma organização que está em um cenário caótico: não realiza testes automatizados, não apresenta comunicação entre as equipes, sem desenvolvimento ágil, demora excessiva para liberação de novas funcionalidades etc. Aqui podemos perceber uma relação importante: ausência de testes automatizados, demora para as entregas e maior

proporção de trabalho não planejado, ou seja, correção de erros de sistema. A literatura está repleta de exemplos desse tipo. Como veremos nos próximos capítulos, sempre que há ausência de testes automatizados, há uma forte tendência de lentidão para liberação de novas funcionalidades e grande quantidade de trabalho não planejado.

Na outra extremidade, podemos verificar o ápice do modelo de maturidade, cujas práticas são relacionadas pela classificação Expert. Nessa categoria, a segurança recebe a relevância necessária, já que sua ausência torna a aplicação vulnerável, podendo levar uma empresa à falência. Temos visto nos jornais um fato que irá entrar nos livros de História: a invasão do Telegram do Ministro da Justiça. É uma falha seríssima, que pode mudar o destino do país inteiro. Não se pode atribuir a culpa somente ao Telegram, pois os hackers entraram na versão web da ferramenta que enviou o número de confirmação ao Ministro. A falha maior foi as operadoras de telefonia permitirem que o Ministro recebesse ligações do seu próprio número de celular, editado por tecnologia Voip, estratégia utilizada pelos hackers para a invasão e captura do código. Não bastaria proibir esse tipo de chamada? É uma verificação de código simples. Com isso, vemos o aplicativo de mensagens perder credibilidade em larga escala. Não é à toa que foi cunhado o termo DevSecOps. Em seu artigo, (HOZ, PAULA DE LA, 2019) dá algumas recomendações de segurança importantes para os desenvolvedores. Por incrível que pareça, a mais simples é o uso de senhas fortes. Sem comentários. Outra dica é o cuidado necessário ao utilizar aplicações ou frameworks de terceiros que podem vir com usuários e senhas padrão. Ao encontrar uma página padrão como /myadmin.php, um hacker pode fazer de tudo para descobrir o usuário e senha, podendo entrar na aplicação. Também aconselha que o desenvolvedor verifique que tipo de texto foi digitado e enviado ao seu sistema, barrando a injeção de scripts. Imagine que, em uma aplicação web, seja possível digitar um conteúdo Javascript em uma caixa de texto que será interpretado pelo navegador quando a página for carregada. Um invasor conseguiria assumir o controle de programação com essa vulnerabilidade. A autora também recomenda que o desenvolvedor sempre faça perguntas, já que algumas vezes pode assumir o risco de uma vulnerabilidade por medo de se expor, simplesmente. É por isso que, neste modelo de maturidade, vemos a segurança ocupar o posto de excelência no que diz respeito à estrutura das equipes: segurança e desenvolvimento devem estar entrelaçados. O Analista de Segurança deve estar próximo ao desenvolvedor.

Ainda na classificação Expert, vemos um ambiente onde a comunicação flui, onde há compartilhamento de ferramentas e informações, equipes mistas, metodologia ágil e entrega contínua, todas características essenciais de DevOps. O conceito Lean pode ser verificado na ausência da necessidade de rollbacks após implantações em produção. É um processo enxuto, pois não há retrabalho: quando um erro ocorre, a falha é aceita e corrigida de maneira natural, sem grandes impactos para a operação. A consequência dessas práticas é um aumento considerável no número de implantações em produção. A presença dos testes automatizados favorece esse cenário, já que as equipes têm menor retrabalho com erros, dedicando mais tempo à novas funcionalidades.

Este modelo, com base em um questionário, permite um diagnóstico rápido do estágio atual de maturidade, envolvendo características bastante diversificadas da metodologia DevOps.

3.1.2 Modelo de Maturidade da Entrega Contínua

É sabido que uma equipe de teste pode perder muito tempo esperando a equipe de desenvolvimento liberar um build. Muitas vezes, a funcionalidade a ser testada está desenvolvida, está no repositório de software, mas não está distribuída, pronta para ser testada. Vamos analisar esta asserção objetiva:

3.1 Modelos de Maturidade *DevOps*: Modelos de Maturidade, Estatísticas e Ferramentas

	BÁSICO	INICIANTE	INTERMEDIÁRIO	AVANÇADO	EXPERT
ESTRUTURA DE EQUIPES	Desenvolvimento, operações, gerenciamento de serviço, segurança e times de qualidade são distintos e independentes.	Equipes de Dev e Ops trabalham juntas para estabelecer a cultura “você constrói, você opera”.	QA está junto com o desenvolvimento para assegurar qualidade nas operações.	Tickets são designados imediatamente ao time de desenvolvimento sem passar pela avaliação de operações.	Segurança está entrelaçada ao processo de desenvolvimento.
COMPARTILHAMENTO	Times de Dev e Ops têm responsabilidades diferentes e seu próprio conjunto de ferramentas. Sofrem para compartilhar dados.	Times de Dev e Ops compartilham algumas responsabilidades, mas ainda utilizam ferramentas diferentes.	Times de Dev e Ops utilizam um conjunto comum de ferramentas, mas compartilham informação manualmente.	Times de Dev e Ops usam um conjunto comum de ferramentas, mas não visualizam o trabalho um do outro.	Somos todos um único time, compartilhando responsabilidades e utilizando um conjunto comum de ferramentas, além de compartilhar informações.
PRIORIZAÇÃO	Ciclo de Planejamento orçamentário (anual, trimestral)	Cadência de planejamento (semanas).	Pequenos projetos e produtos iterativos	Planejamento contínuo	Experimentação e releases contínuos
AMBIENTE DE TESTES	Testes manuais com ambientes de testes separados do desenvolvimento	Testes unitários	Testes integrados automatizados	Ambiente de teste hostil	Teste de carga redirecionando transações de produção para um ambiente de teste
ESTRUTURA ORGANIZACIONAL PARA ENTREGAS	Desenvolvimento coloca para funcionar e sai de cena para que Operações finalize o trabalho	Engenheiros de Operações fazem parte dos times de Dev	Temos apenas um time, não dois	Nossa arquitetura está projetada para operações e escalabilidade	Gerenciamento de produto escreve User Stories com operações em um nível mais elevado
RELEASE	Integração contínua, enviando código ao Master Branch	Entrega contínua para pré-produção	Processo tradicional de release (Cascata)	Deploy contínuo para produção	Deploy contínuo para produção reforçado com releases experimentais
FREQUÊNCIA DE IMPLANTAÇÃO	Em poucos meses	Mensalmente	Semanalmente	Diariamente	Muitas vezes por dia
IMPLANTAÇÃO	Releases de novas aplicações têm muitos problemas em produção, forçando rollbacks	Novas aplicações têm problemas ocasionais, sendo necessárias correções em produção	Integração contínua revela potenciais problemas antes do release	Deploy contínuo possibilita rollback rápido para remediar problemas operacionais	Entrega e melhoria contínuas previnem a necessidade de rollbacks
IMPLEMENTAÇÃO x CORREÇÃO	Predominantemente trabalho não planejado, raramente desenvolvimento de novas funcionalidades	Trabalho não planejado na maior parte das vezes, uma ou outra nova funcionalidade	Equilíbrio entre trabalho não planejado e novas funcionalidades	Novas funcionalidades na maior parte das vezes, algum trabalho não planejado	Predominantemente novas funcionalidades, raramente trabalho não planejado
VELOCIDADE DOS TESTES	Não são executados testes	1 dia ou mais	1 hora a um dia	10 minutos a uma hora	10 minutos ou menos

Tabela 3.1: Modelo de Maturidade *DevOps* - Atlassian.

o deploy em sua equipe é feito independentemente de um release. Essa afirmação é uma das métricas do Modelo de Maturidade de Entrega Contínua proposto por (REHN, A. AND PALMBORG, T. AND BOSTROM, P., 2019). Os autores, apresentam o artigo como um caminho a seguir para se atingir a entrega contínua. Na tabela 3.2, podem ser vistas as métricas do modelo em cada quadrante. Em uma estrutura muito semelhante ao modelo anterior, as linhas representam a categoria e as colunas representam o seu estágio de evolução.

Em comparação com o modelo Atlassian, o nível Básico parte de um ponto mais avançado. Possui características de Integração Contínua e Testes Automatizados, ainda que em pequena escala. É comum encontrar nesse estágio a presença de aplicações legadas monolíticas, difíceis de serem escaladas e mantidas, devido ao seu nível de acoplamento. Durante a evolução, essa aplicação deverá ser “quebrada” em componentes menores, ou ser depreciada.

O nível Iniciante apresenta uma mudança relevante na cultura, apresentando uma aproximação entre o desenvolvimento e os testes, iniciação em Agile e um espírito de equipe onde os problemas são vistos como algo a ser resolvido de forma coletiva.

No próximo nível na escala de evolução, o Intermediário, ocorre o maior ponto de mudança deste modelo. Ocorrem mudanças significativas de cultura, como a remoção de barreiras entre Dev e Ops e o crescimento da autonomia. A arquitetura melhora com a chegada de configuração como código e com o versionamento dos fontes no repositório por abstração, em vez de criar vários branches. Isso facilita a integração contínua, já que reduz a quantidade de merge de código que precisa ser feita para se obter um build. É nesta fase que ocorre a maior automatização, surgindo o primeiro pipeline de entrega contínua, direto para produção.

No nível seguinte, o Avançado, destaca-se a categoria de testes. São automatizados os testes de aceitação com grande abrangência, os testes de segurança e os testes de performance. Como estratégia, alguns testes de funcionalidades críticas são conduzidos manualmente, podendo-se utilizar a técnica de exploração, procurando por erros seguindo caminhos indiretos e complexos que o software pode assumir. Como grande avanço destaca-se a realização do deploy com tempo zero de parada, de apenas um componente da aplicação, independentemente dos demais.

Por último, o estágio Expert destaca-se pelo alto nível de automação, com deploys ocorrendo sem intervenções para produção e informação sendo gerada em tempo real. Aqui é proposta a medição do resultado trazido para o negócio de cada funcionalidade implementada.

Este modelo é abrangente, com uma gama diversa de conceitos pertinentes à metodologia DevOps. Além de se apresentar como um bom diagnóstico do estágio atual de equipes de desenvolvimento de software, mostra um caminho a ser seguido na busca da entrega contínua.

3.2 Estatísticas DevOps

Como complemento dos modelos acima, podemos citar o trabalho realizado por Puppet: State of DevOps Report 2018 (PUPPET, 2019). Para a elaboração desse estudo, mais de 30.000 profissionais, principalmente dos Estados Unidos e da Europa, responderam o questionário. Segundo o relatório, das organizações mais evoluídas em DevOps:

- 47% estão 24 vezes mais propensas a tornar o monitoramento configurável pelo próprio time que acompanha a aplicação em produção.
- 46% estão 23 vezes mais inclinadas à reutilização dos padrões de deploy.
- 44% estão 44 vezes mais propensas à reutilização de padrões de teste.

3.2 Estatísticas DevOps: Modelos de Maturidade, Estatísticas e Ferramentas

	BÁSICO	INICIANTE	INTERMEDIÁRIO	AVANÇADO	EXPERT
Cultura e Organização	- Trabalho não priorizado - Processo definido e documentado - Commits frequentes	- Um backlog por equipe - Equipes estáveis - Uso de métodos ágeis básicos - Remoção de barreiras entre dev e teste - Dor compartilhada quando problemas ocorrem	- Colaboração em equipe estendida - Propriedade de componente gera autonomia - Remoção de barreiras entre Dev e Ops - Processo comum para todas as mudanças - Decisões descentralizadas	- Equipe de ferramentas - Responsabilidade da equipe até produção - Deploy não depende de release - Melhoria contínua	- Equipes multifuncionais - Sem rollbacks em produção
Design e Arquitetura	- Plataforma e tecnologias consolidadas - Presença de aplicações monolíticas	- Sistema organizado em módulos - Gerenciamento de API - Gerenciamento de bibliotecas - Controle de versão em mudanças de banco	- Sem branching, ou mínimo - Branch por abstração para reduzir branching e facilitar CI - Configuração como código - Feature hiding	- Arquitetura totalmente baseada em componentes - Deploy e release individual de cada componente - Melhorias métricas do negócio	- Infraestrutura como código
Build e Deploy	- Código fonte versionado - Builds por script - Builds básicos agendados (CI) - Servidor de build dedicado - Deploy manual documentado	- Polling builds (build incremental) - Builds armazenados facilitam gerenciamento de dependências - Tag manual dos builds e versionamento - Primeiro passo para deploys padronizados	- Build auto iniciado por commit - Tag automatizado e versionamento - Build único para deploy em qualquer lugar - Fila automatizada de mudanças em banco - Pipeline básico com deploy para produção - Mudanças de configuração por script - Processo padronizado para todos os ambientes	- Deploy com tempo zero de parada - Múltiplas máquinas de build - Deploy completo de build de banco	- Forno de builds - Deploy contínuo sem intervenções
Teste e Verificação	- Testes unitários automáticos - Ambiente de teste separado	- Testes integrados automáticos	- Testes isolados automáticos de componente - Alguns testes automáticos de aceitação	- Testes de aceitação automatizados de abrangência ampla - Testes automatizados de performance - Testes automatizados de segurança - Testes manuais baseados em riscos	- Medição do valor da alteração para o negócio
Informação e Relatórios	- Métricas de processo como referência - Relatório manual	- Medida do processo - Análise estática do código - Relatórios de qualidade agendados	- Modelo de informação comum - Rastreabilidade no pipeline - Histórico de relatórios	- Gráficos como serviço - Análise dinâmica da cobertura do teste - Relatório de análise de tendências	- Gráficos dinâmicos e dashboards - Análise transversal de silos, cruzando fronteiras da organização

Tabela 3.2: Modelo de Maturidade da Entrega Contínua

- 44% estão 44 vezes mais inclinadas à contribuir com melhorias em ferramentas providas por outras equipes.
- 53% estão 27 vezes mais propensas à utilização de ferramentas de gerenciamento de configuração.

Esse estudo trouxe uma contribuição interessante: há uma divergência entre a percepção dos executivos das organizações com relação à percepção das equipes. Os primeiros tendem a superestimar a evolução em DevOps. A pesquisa revelou que 64% de executivos que responderam o questionário acreditam que times de segurança estão envolvidos em projeto e deploy, enquanto apenas 39% de membros das equipes têm essa visão. Essas estatísticas reforçam a contínua tendência para o crescimento das práticas DevOps como mudanças na cultura, maior integração e autonomia das equipes, priorização da segurança, aumento do compartilhamento e reutilização.

Abaixo, podemos avaliar um outro trabalho relevante: xMatters Atlassian DevOps Maturity Survey Report 2017 (XMATTERS, ATLASSIAN, 2019). A pesquisa é composta de uma amostra de cerca de 1000 participantes. Do mesmo modo que os modelos apresentados neste trabalho, o levantamento também avaliou os participantes classificando-os de nível básico a avançado.

- 81% responderam que os responsáveis por desenvolvimento e operações compartilham algumas ferramentas, classificando-se em nível avançado.
- 29% responderam que o conhecimento é compartilhado na organização somente quando requisitado, classificando-se em nível iniciante.
- 41% reportam que aplicações implantadas têm pequenos incidentes em produção, exigindo correções.
- Mais de 50% realizam monitoramento de Aplicações, Serviços, Infraestrutura, Transações e Experiência do Usuário. Percebe-se que o alto monitoramento de Aplicações e Infraestrutura não diminui significativamente os incidentes para o usuário final.
- Cerca de 50% responderam que, durante o processo de QA, ocorrem Testes integrados, Testes manuais em ambientes separados, Teste de carga, Testes automatizados e Testes unitários. Uma possível explicação para o alto número de problemas em produção seria o baixo índice de Testes automatizados.
- 61% reportaram que as soluções de monitoramento predizem incidentes potenciais antes que os usuários sejam afetados.
- 65% dos entrevistados responderam que DevOps os conduz aos resultados esperados.

Neste último item, podemos perceber que 35% dos entrevistados não estão satisfeitos com DevOps. É uma análise difícil de ser feita, mas o levantamento dos motivos precisa ser caso a caso. Algumas vezes, os fatores culturais, como resistência à mudança podem dificultar muito. Outra dificuldade é que algumas vezes pode haver mais de um caminho correto a ser seguido, gerando dúvidas e até mesmo fracassos, causados pela escolha errada para o momento. Contudo, a metodologia está consolidada, com muitos casos de sucesso. Dessa forma, cada organização deve passar pelo processo de experimentação contínua para conseguir implementar a metodologia em seu caso concreto.

3.3 Ferramentas DevOps

Em busca de um modelo de excelência de Entrega Contínua, há muitas práticas que uma organização precisa realizar. Em um cenário de evolução constante, assim como ocorre no desenvolvimento de software, muitas ferramentas têm surgido para permitir que uma companhia pratique os conceitos envolvidos e consiga dar o passo necessário em sua evolução de maturidade de processo DevOps. Há

muitas ferramentas, muitas vezes oferecendo as mesmas funcionalidades do que outras. O que pode diferenciar é o fato de serem gratuitas ou pagas. A tendência das primeiras é uma complexidade maior de configuração, exigindo mais tempo para que uma equipe consiga implantá-la. Também costumam exigir uma curva de aprendizado maior. Já as ferramentas pagas possuem rotinas que permitem iniciar a operação mais rapidamente, reduzindo tempo gasto com implantação e aprendizado.

Na tabela 3.3 foram selecionadas algumas ferramentas, tanto gratuitas quanto pagas, por critérios de popularidade, o que leva a uma maior comunidade de usuários, consequentemente de maior suporte, documentação etc. O quadro está organizado em categorias importantes do processo DevOps e Entrega / Melhoria Contínuas. Quem pretende seguir a carreira de Engenheiro de DevOps pode utilizar essa lista como ponto de partida para se especializar. Durante sua leitura é importante tentar aplicar conceitos aprendidos até aqui.

A ferramenta Jenkins merece destaque por ser livre, de Integração e Entrega Contínuas, além de ser bastante popular. Possui um ecossistema vasto de plugins permitindo integração com diversas ferramentas. Na imagem 3.2, podemos verificar o pipeline proposto pela ferramenta. O processo é iniciado por um commit realizado no repositório. Jenkins, em sua integração com Git, dispara o processo de release para produção após verificar em seu monitoramento um commit realizado pelo desenvolvedor. Em seguida, é realizado checkout do repositório e são coletados os repositórios dependentes. A ferramenta dispara integração com Docker para realizar os builds em paralelo de cada componente. No estágio seguinte, são realizados, também em paralelo, testes automatizados de integração, de dependências, Smoke Tests para garantir que o build correu bem, além de testes end-to-end que avaliam o funções do sistema de ponta a ponta. O deploy é realizado, o processo termina e a correção pode ser enviada automaticamente para a produção, dependendo do nível de maturidade atual da organização.

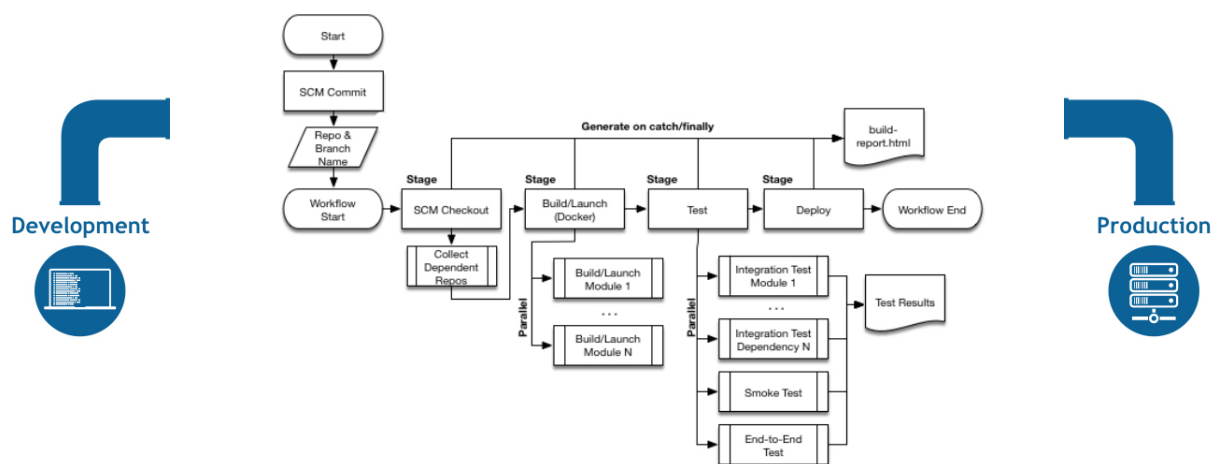


Figura 3.2: Pipeline Jenkins de Entrega Contínua

Fonte: Adaptada de (JENKINS, 2019).

3.3 Ferramentas populares na metodologia *DevOps*

CATEGORIA	FERRAMENTAS	CARACTERÍSTICAS
Build	Gradle	- Build incremental compila somente o que não foi alterado. - Velocidade.
Gerenciamento de Código Fonte (Repositório)	Git	- Controle de diferentes versões do software. - Restauração de versões anteriores.
Hospedagem de repositórios	Bitbucket, GitHub	- Código fonte na nuvem. Integração com Slack. - Alerta de novos commits.
Comunicação	Slack	- Ferramenta bastante difundida entre desenvolvedores. - Viabiliza a cultura DevOps de comunicação.
Servidor de integração e entrega contínuas	Jenkins, Bamboo (Atlassian)	- Status de cada etapa do Pipeline. - Ecossistema de plugins. - Integração com dezenas de ferramentas DevOps, inclusive Docker e Puppet.
Containerização	Docker	- Deploy automático de aplicações. - Aplicação resultante sem dependências, de fácil execução. - Independência de sistema operacional e de plataforma. - Abstração do ambiente em tempo de execução da aplicação. - Facilita migração entre nuvens. - Permite arquitetura baseada em componentes, diminuindo dependências.
Plataforma de gerenciamento de containers	Kubernetes, Titus, Amazon ECS (Elastic Container Service)	- Orquestração de containers. - Integração com Docker. - Permite automatização do gerenciamento de centenas de containers. - Escalabilidade e disponibilidade de grandes aplicações distribuídas.
Gerenciamento de configuração / Infraestrutura como código	Puppet, Ansible	- Permite levantar rapidamente o que precisa ser alterado em uma atualização. (Endereços de máquinas / Diretórios / Constantes etc) - Aborta processo quando encontra falha de configuração. - Integração com ferramentas DevOps.
Monitoramento de infraestrutura	Nagios, SolarWinds	- Monitoramento de performance. - Detecção de falhas. - Indicadores e Dashboards.
Monitoramento de performance da aplicação	Dynatrace	- Monitora performance específica da aplicação. - Monitora velocidade da aplicação em partes específicas do código.
Gerenciamento de Incidentes	PagerDuty, VictorOps	- Processo de Gerenciamento de Incidentes
Rastreio de erros	OverOps	- Envia stacktrace e código fonte de produção que obteve erro com o estado das variáveis.
Qualidade do Código	CodeClimate, SonarQube	- Análise contínua da qualidade do código. - Com o tempo, torna o código manutenível. - Integração com GitHub.
Detecção de Intrusões	Snort, SolarWinds	- Detecção de tráfego malicioso.

Tabela 3.3: Ferramentas populares na metodologia *DevOps*

3.4 Estudo de Caso - Netflix

A trajetória da Netflix em direção à DevOps iniciou-se durante uma crise: em 2008 seu banco de dados passou por um processo de corrupção. Na época, a empresa ainda enviava DVDs pelos correios e seus clientes ficaram três dias com o serviço indisponível devido ao incidente. Este estudo de caso, descrito por (AMIDO, 2019), apresenta o cenário da evolução da corporação rumo à DevOps. Outro fator que forçou a companhia a melhorar sua infraestrutura tecnológica, sendo uma das precursoras do desenvolvimento de conceitos importantes de DevOps como Microserviços e Containerização, foi a urgência de escalabilidade. A organização começou a oferecer o serviço de streaming em 2007 e, em 2015, era responsável por 36% de downstream na América do Norte.

Uma hipótese para o colapso ocorrido em 2008, pode ter sido a sobrecarga do ambiente monolítico, verticalizado. A urgência em escalar pode ter forçado essa situação. Como a oportunidade de mercado era gigantesca, a equipe tecnológica foi forçada a encontrar uma saída. Isso fez com que a Netflix fosse uma das primeiras a adotar Microserviços, transformando uma aplicação vertical, baseada em um banco de dados em um Data Center, em um sistema distribuído em nuvem, horizontalizado. A inteligência do processo está em adicionar mais máquinas horizontalmente para escalar a capacidade de atendimento, em vez de adicionar mais recursos, como processamento e memória, em uma única máquina que pode falhar e parar todo o atendimento. Em um cenário horizontalizado, uma máquina que falha pode ser rapidamente substituída. Este processo, que também inclui a migração para a nuvem, iniciou-se em 2008 e terminou em 2016.

Essa estratégia de transferência da responsabilidade de escalabilidade para a nuvem, gerou uma vantagem competitiva importante para a empresa. Com uma arquitetura horizontalizada, o cenário de crescimento de 50% em seis meses pelo qual a Netflix passou, ficou sob controle, apenas exigindo ações operacionais de planejamento. Também permitiu tratar com naturalidade o fato de saber que haveria picos de utilização, que iria requerer mais recursos da nuvem em determinados períodos. Com isso, a organização teve condições de focar na liberação de novas funcionalidades, criando um produto cada vez melhor.

Em abril de 2017, a companhia atingiu a incrível marca de 1.000.000 de containers publicados por semana. É um cenário tecnológico bastante diferente daquele de 2008. A companhia jamais teria conseguido escalar sua capacidade com aquele cenário. Essa marca foi sustentada por uma arquitetura baseada em Docker e Titus, uma plataforma de gerenciamento de container de autoria da Netflix, que tornou-se open source em abril de 2018. A empresa precisou desenvolver este produto, já que possuía requisitos de containerização específicos de seu negócio. A ferramenta possui integração com Amazon ECS, produto da Amazon, também dedicado à orquestração de containers.

Outro fator relevante para a estabilidade dos serviços de streaming foi a estratégia corporativa de construir para falhar. O objetivo é trazer a falha para o dia a dia, de forma que, quando realmente aconteça um problema, não ocorra paradas, já que em 2014, uma hora de inoperância custava US\$ 200.000 à organização. Pensando nisso, foi desenvolvido o Chaos Monkey, que também tornou-se Open Source. Esse software derruba servidores na nuvem, para testar estabilidade do ambiente de produção. Essa foi a forma encontrada para se proteger de problemas como o ocorrido na véspera de Natal de 2012, quando a Netflix ficou com o serviço indisponível por várias horas, devido a incidentes na nuvem.

Os principais benefícios trazidos pela cultura DevOps para a Netflix foram releases frequentes de software confiável e um alto nível de automação. Somado a isso, a metodologia trouxe a estrutura de equipe e conceitos necessários para superar as barreiras que impediam a escalabilidade, juntamente com a estratégia que permitiu aceitar as falhas como parte do processo, sem impacto e sem paradas

de sistema.

3.5 Conclusões

Quem acessou o Twitter recentemente, recebeu a mensagem da figura 3.3. É um aviso de novas funcionalidades. As grandes empresas de TI estão na frente do processo DevOps, com um excelente desempenho no que se refere à entrega contínua. Sabemos do desafio que está por trás disso e quantos passos precisam ser dados para atingir a maturidade no processo.

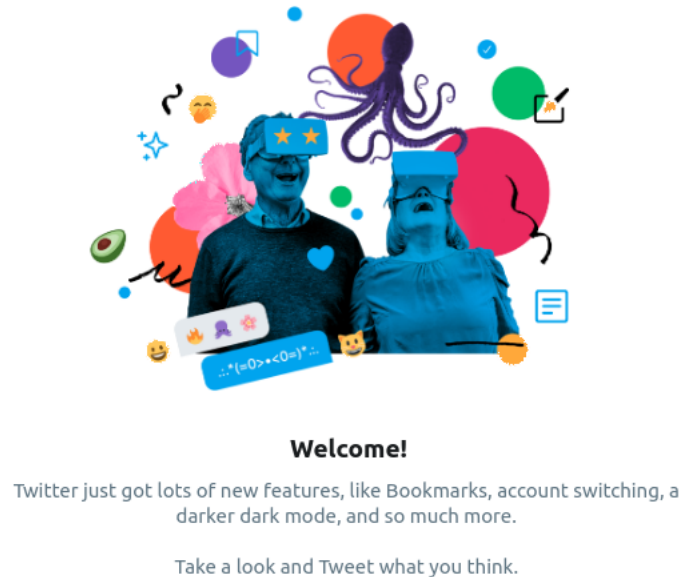


Figura 3.3: Mensagem de novas funcionalidades do Twitter

Ver uma mensagem como essa com frequência, ou receber várias atualizações sucessivas de aplicativos, pode significar muitas coisas. Por exemplo, a existência de testes automatizados no processo de entrega contínua que reduzem o trabalho não planejado e aumentam a entrega de novas funcionalidades.



4. Testes: Conceitos relevantes

Neste capítulo, iremos abordar os principais conceitos e tipos de teste. Ao final, espera-se que o estudante possa:

- Compreender o conceito de Cobertura de Testes.
- Compreender que um defeito de software nem sempre é descoberto facilmente.
- Conhecer diferentes estratégias de teste de software, de acordo com uma coletânea de possíveis testes.
- Compreender a influência do Desenvolvimento Orientado a Testes na arquitetura do sistema.
- Diferenciar testes funcionais de testes não funcionais.

Teste de Software

Sabemos que a atividade de desenvolvimento de software é complexa. Por mais que haja rigor na especificação e no desenvolvimento, não é possível percorrer todos os cenários possíveis que a execução pode seguir. São milhões de combinações diferentes que as variáveis do código podem assumir, sendo impossível eliminar situações de erro. Por isso vemos essa área do conhecimento crescer tanto, trazendo cada vez mais nomenclaturas e abordagens diferentes. Na busca pela excelência da qualidade do software, o teste possui um papel fundamental. O teste de software é uma atividade que visa a verificar se os requisitos especificados, tanto funcionais, quanto não funcionais, são atendidos na maioria dos cenários, não somente nos mais simples, mas também nas exceções. Além disso, as funções devem ser executadas com qualidade, provendo a melhor experiência para o usuário.

Defeito de Software

Defeito, erro ou bug é um comportamento inesperado do software intrínseco ao seu desenvolvimento. Pode se manifestar sempre que um código é executado ou em determinada combinação de condições, dificultando que seja reproduzido e corrigido.

```
1 public class Vetores {  
2
```

```
3      String retornaElemento(String[] a, int n) {  
4  
5          return a[n];  
6  
7      }  
8  
9  }
```

Código Fonte 4.1: A classe Vetores e a função *retornaElemento*

A função *retornaElemento*, contida na classe representada pelo código fonte 4.1, que retorna o elemento *n* do array *a*, possui pelo menos quatro bugs que são manifestados quando:

- O objeto *a* está nulo
- O objeto *a* está vazio
- *n* é negativo
- *a* não possui o elemento referenciado por *n*

É curioso pensar que a função *retornaElemento* funciona perfeitamente em boa parte de combinações e valores que serão passados a ela. Contudo, provavelmente o usuário do sistema obterá um erro nas quatro situações acima.

Um defeito deve ser bem identificado no momento em que ocorre, pois isso viabiliza a correção. O erro pode ocorrer na presença do desenvolvedor, durante a execução dos testes ou até mesmo após a implantação. O primeiro cenário é mais favorável, pois o registro e a correção podem ser conduzidos facilmente. Quando o erro ocorre com o usuário, é necessário documentar o problema para que chegue até o desenvolvedor, de forma a ser possível reproduzir o problema. Quanto mais complexo o sistema, melhor deve ser essa gestão. Quanto mais as fases do projeto avançam, mais custosa fica a correção de um novo bug encontrado.

Caso de Teste

Um caso de teste é uma verificação para conferir se uma funcionalidade concreta do software funciona como esperado. Geralmente, são organizados nos chamados Cadernos de Teste.

Cobertura de Teste

Referência para saber em que medida um software foi testado. Algumas dessas referências podem ser:

- Número de casos de teste que foram executados, com relação ao total de casos de teste elaborados.
- Número de linhas de código que foram executadas, em relação ao total de linhas.
- Número de nós do código que foram executados, em relação ao total de nós.

Complexidade Ciclomática

Na função Java 4.2, para que todos os nós do programa sejam executados, o teste deve forçar que a expressão *a > b* seja avaliada tanto como verdadeira quanto como falsa, garantindo uma cobertura de 100%.

Combinando esses nós, é possível medir o número de possíveis caminhos que um software pode executar. Conhecida como complexidade ciclomática, essa técnica permite definir a complexidade do programa. No caso abaixo, a complexidade ciclomática é igual a 2, pois há apenas dois caminhos que podem ser percorridos. É importante que o desenvolvedor tenha em mente que essa complexidade

deve ser a mais baixa possível. Isso reduz esforço com testes e tende a aumentar a cobertura, já que diminui o número de possíveis caminhos.

```
1 boolean aMaiorQueB(int a, int b) {  
2  
3     if (a > b) {  
4         return true;  
5     }  
6  
7     return false;  
8  
9 }
```

Código Fonte 4.2: Função simples para avaliação da Complexidade Ciclomática

Stub

Muitas vezes, durante o desenvolvimento, uma funcionalidade não estará preparada para ser testada se depender de funções do código ainda não implementadas. Um stub é uma versão fixa e simplificada do código ainda não desenvolvido.

Mock

Um mock tem uso similar ao do stub, mas sua complexidade vai mais além. Existem mocks prontos que simulam uma API inteira, como é o caso do framework Spring. Essa ferramenta disponibiliza mocks da API JNDI, utilizada como base para serviços de diretório como LDAP e DNS, e também para a API de servlets do Java. Um mock, pode simular uma base de dados inteira, facilitando o processo de teste.

4.1 Abordagens, Modalidades e Tipos de Teste

4.1.1 Abordagem Caixa Branca

Na abordagem de Caixa Branca, o código fonte do software é levado em consideração durante os testes. Dessa forma, é necessário ter conhecimento de programação para realizá-lo. O Analista de Testes avalia o código pensando em uma combinação de condições não prevista que possa levar o software a falhar.

4.1.2 Abordagem Caixa Preta

Basicamente, consiste em testar o software sem analisar o código. O teste de Caixa Preta muitas vezes é citado como teste funcional, avaliando se o produto atende aos requisitos definidos na especificação.

4.1.3 Abordagem de Regressão

O teste de regressão é utilizado para saber se o software teve seu comportamento alterado após uma mudança. Os testes automatizados se enquadram nesta abordagem.

4.1.4 Testes Funcionais

Testes Funcionais são aqueles que visam a conferir o funcionamento de requisitos específicos do sistema. No caso de uma aplicação de Internet Banking, um teste funcional seria verificar se uma segunda via de boleto pode ser emitida corretamente, por exemplo.

4.1.5 Testes Não funcionais

Testes Não Funcionais são aqueles que não estão relacionados a requisitos do negócio da aplicação. Dentre os mais importantes, podemos destacar: Segurança, Performance, Usabilidade, Escalabilidade etc.

4.1.6 Teste End-to-end

Esta técnica define o teste que abrange um processo da aplicação do início ao fim, considerando também as integrações com Bancos de Dados, Web Services etc.

4.1.7 Análise de Mutantes

A Análise de Mutantes é uma estratégia de teste cujo objetivo principal é maximizar a cobertura. Isso é feito criando pequenas mudanças no código gerando versões diferentes do programa testado: um programa mutante. Ao executar o programa mutante, a saída é comparada com o programa original. Se as saídas forem diferentes, o mutante é considerado morto. A cobertura do teste é calculada dividindo-se o número de mutantes mortos pelo total de mutantes. Conforme os testes avançam, alguns mutantes permanecem vivos, requerendo uma análise mais detalhada para saber o motivo de retornarem a mesma saída. Isso acaba exigindo uma análise minuciosa do código pelo Analista de Testes, até ser decidido se o mutante sempre terá a mesma saída, sendo considerado equivalente. Dessa forma, a cobertura do teste aumenta confiavelmente.

4.1.8 Teste Unitário

Os Testes Unitários são pequenas verificações dentro de um método de um programa. Seu objetivo é documentar como este método deve se comportar e emitir um erro quando isso não ocorrer. Geralmente é um teste automatizado, o mais difundido entre eles, já que é de fácil elaboração e de rápida execução, trazendo um bom retorno do investimento.

4.1.9 Teste Integrado

O teste integrado está bastante relacionado ao teste unitário na forma de ser escrito e executado, contudo seu foco é a interface dos métodos do programa.

4.1.10 Testes de Aceitação do Usuário

Os Testes de Aceitação do Usuário, do inglês User Acceptance Testing - UAT, são realizados para conferência do cliente ou da área da empresa demandante do software. Também conhecidos em algumas empresas como homologação, esses testes avaliam se o software se comporta de acordo com as regras de negócio e critérios de aceitação. São importantes, pois, muitas vezes o software é muito específico e o usuário final, por trabalhar e dominar o conhecimento de sua área, pode realizar a validação com maior assertividade.

4.1.11 Smoke Testing

Também conhecido como Build Verification Test, Smoke Testing são validações realizadas no software para saber se há restrições que impedem a publicação de um novo release. Dependendo do estágio de organização dentro de uma fábrica de software, um desenvolvedor pode sobrescrever alterações realizadas anteriormente por outro. Para o cliente isso é percebido quando as funcionalidades ou correções adicionadas recentemente deixam de existir após uma implantação. Dependendo da estratégia de teste, a utilização de Smoke Testing pode solucionar o problema. Uma verificação comum de Smoke Testing é, testar se, após o build, a aplicação inicializa sem problemas. Uma das formas de verificar isso é saber se o usuário consegue realizar login. Uma prática comum de automatização desse teste em linguagem Java é a utilização das ferramentas Cucumber, Selenium e Junit, integradas na IDE Eclipse. A ferramenta Cucumber cria os casos de teste com base na User Story. Para cada passo da User Story, a ferramenta Selenium é acionada, simulando o processo abrindo o navegador automaticamente e realizando os comandos do teste. Ao final, se a página de login é aberta e uma mensagem de boas vindas aparece, então conclui-se o teste com sucesso. A ferramenta JUnit é utilizada para disparar o processo e acompanhar o resultado de cada caso de teste.

4.1.12 Testes Exploratórios

Esses testes estão mais relacionados à capacidade dos Analistas de Testes buscarem situações que podem levar a problemas no software. O trabalho desses profissionais exige essa habilidade, um modo de pensar bastante crítico, que não se guia pelas situações padrão. Auxiliam a reduzir erros que são descobertos pelos próprios usuários, já no ambiente de produção.

4.1.13 TDD - Test Driven Development

Num primeiro momento é bastante curioso pensar que esta técnica exige que o desenvolvedor escreva os testes antes do programa. Contudo, é uma prática que ganhou bastante visibilidade e adesão pelos seus benefícios. Essa abordagem obriga o desenvolvedor a planejar o código que vai escrever: de outra forma, poderia ser impossível testá-lo. Antes da disseminação do padrão de implementação MVC, que separa as camadas de aplicação em três (Model, View and Control), era comum encontrar aplicações que mesclavam essas três camadas. Nesse cenário, como não se pode separar os diferentes componentes, escrever um teste fica inviável. Essa prática é conhecida por melhorar a qualidade do código, e, conseqüentemente, reduzir defeitos do software, possibilitando que mais entregas sejam realizadas.

4.1.14 BDD - Behaviour Driven Development

Assim como TDD, o desenvolvimento orientado a comportamento do software, BDD, é oriundo da Metodologia Agile. Neste modelo, os testes também são escritos antes do desenvolvimento. Contudo, esses testes são derivados da própria User Story, ou seja, da própria especificação de uma funcionalidade. Para entender melhor, em Agile, segue um exemplo de User Story para um aplicativo de organização pessoal:

Story: Criar tarefa com lembrete

Como um usuário

Eu quero criar uma tarefa

De forma que eu possa ser avisado quando chegar o horário

A partir da User Story, o teste pode ser escrito com base em alguma ferramenta para esta finalidade. Uma delas, bastante difundida, chama-se Cucumber. Esses testes são escritos em

linguagem de alto nível, humanamente compreensível.

4.1.15 Especificações Executáveis

Bastante relacionado à BDD, as Especificações Executáveis realizam uma tarefa bastante ousada. Vinculam as histórias de usuário - escritas em linguagem de compreensão comum entre clientes, analistas e desenvolvedores - com casos de teste. A especificação pode ser literalmente executada, rodando os testes, inclusive.

A ferramenta Cucumber realiza essa tarefa utilizando um artifício chamado step: para cada particularidade da história do usuário, associa o teste a ser executado. Esses testes simulam a carga de páginas web, cliques em botões etc.

Cucumber utiliza uma DSL (Domain Specific Language) bastante adotada, chamada Gherkin. Isso viabiliza a linguagem comum entre os envolvidos no processo de desenvolvimento, agregando um valor considerável.

Mais uma vez, fica visível como a rede de testes automatizados, proposta por HIBBS et al pode ser utilizada como documentação viva do sistema.

4.1.16 ATDD - Acceptance Test Driven Development

Está técnica é similar à BDD, contudo se diferencia por trabalhar apenas com os testes necessários para garantir que uma aplicação possa ser aceita pelos solicitantes.

4.1.17 Teste A/B

Esta técnica avalia os resultados trazidos por duas versões diferentes de um software, permitindo avaliar qual delas traz o melhor resultado final, por experiência, ou seja, empiricamente. Entre essas duas versões, deve haver apenas uma única característica diferente. A versão A apresenta as características do ambiente de produção atual. A versão B contém a mudança. Este teste é bastante utilizado em páginas web, para análise do comportamento de usuário, durante os testes de experiência e usabilidade. Permite saber em qual versão o usuário interage mais, evidenciando uma melhor jornada.

4.2 Casos de Falhas em Software

Cada vez mais a nossa segurança tem dependido de bons testes. A tendência é que isso aumente em grandes proporções. Hoje, os aviões já pousam sozinhos, controlados por dispositivos e programas. Não é à toa que a cobertura de testes é avaliada para sistemas de aviação e até mesmo para veículos, de modo a garantir o padrão de confiabilidade adequado.

Em breve os carros autônomos estarão circulando em grandes quantidades pelas ruas. A empresa Tesla pretende produzir mais de meio milhão de táxis autônomos até 2020. Esse anúncio, que inicialmente era de um milhão de veículos, foi dado uma semana antes da companhia ter sido processada pela morte de um engenheiro da Apple que dirigia um de seus veículos, conforme reportado em (GIZMODO, 2019). Em uma rodovia, o carro estava no piloto automático e o motorista estava sem as mãos no volante. Depois de receber alguns alertas sonoros que não foram respondidos, o veículo acelerou e esterçou o volante para o muro de contenção automaticamente.

Um erro chamado Condição de Corrida provocou dezenas de acidentes, alguns deles fatais. Em 1985, uma máquina de radiação canadense chamada Therac-25 matou 3 pacientes exatamente por causa dessa falha (LEVESON; TURNER, 1993). Uma das formas desse bug ocorrer é que

múltiplas threads de um programa tentem atualizar a mesma variável, no mesmo momento. Isso faz com que somente uma delas atualize o valor, de fato. Se esta variável é utilizada para medir a radiação disparada, por exemplo, e não é incrementada de acordo com o ocorrido, instala-se uma Condição de Corrida, onde a variável permanece com o valor baixo, enquanto está sendo liberada grande quantidade. Esse mesmo erro ocorreu em 2003 no Sistema de Gerenciamento de Energia da corporação GE (POULSEN, 2004), quando três linhas de energia com problema foram desarmadas no mesmo momento. O bug que não havia sido descoberto no código foi o responsável pelo apagão norte-americano de 2003, afetando alguns estados do Nordeste. A falha evitou que alarmes fossem disparados aos empregados que monitoravam o sistema. Algumas localidades ficaram sem energia por dois dias. Mais tarde, ao descobrir o bug, a empresa divulgou uma correção no software.

Uma falha ocorrida este ano ocasionou um depósito por engano de R\$ 77.000.000,00 na conta bancária de um curitibano (GAZETA DO POVO, 2019). Depois de avisar as autoridades, ele tentou pagar alguns boletos com sua parte do dinheiro, mas obteve erro. Quando o problema foi regularizado, todo o dinheiro da conta dele foi removido, causando ainda mais transtornos.

Falhas desse tipo, em proporções menores, não são tão difíceis de acontecer e atentam diretamente contra a credibilidade de empresas e pessoas. Pequenos erros repetidos, podem tornar um sistema conhecido mundialmente, de forma pejorativa. Essas falhas reforçam a necessidade de testes abrangentes, cheios de criatividade, com emprego de técnicas distintas que garantam uma maior cobertura. Conhecer diferentes categorias de defeitos e classificá-los pode ser uma estratégia interessante de se ampliar estratégias de teste.

4.3 Conclusões

Neste capítulo, abordamos os principais conceitos e tipos de teste. Foram abordados conceitos importantes como a Cobertura de Testes, como uma métrica importante para saber o quanto o software foi testado. Vimos que nem sempre um defeito de software é descoberto facilmente e sua presença no código pode causar problemas em grande escala ou denegrir a imagem de uma organização ao longo do tempo. Pudemos avaliar diferentes estratégias de teste de software, divididas em caixa branca, ou caixa preta, funcionais ou não funcionais, manuais ou automáticos. Também exploramos inicialmente o Desenvolvimento Orientado a Testes, que tem se mostrado uma excelente prática, tornando melhores a arquitetura do software e a qualidade do código, facilitando os testes.



5. Testes Automatizados

Neste capítulo iremos aprofundar nos Testes Automatizados. Serão apresentados dois estudos de caso, mostrando o que essa abordagem pode realizar, transformando um cenário de poucas entregas de novas funcionalidades. Também serão apresentadas duas provas de conceito. Ao final, é esperado que o leitor possa:

- Ter uma visão geral das ferramentas, de suas categorias e de seus requisitos atendidos.
- Descrever a relação entre testes automatizados, trabalho não planejado e entrega de novas funcionalidades.
- Compreender como os testes automatizados podem se tornar documentação do sistema.
- Conhecer um cenário real de desenvolvimento TDD.
- Conhecer um cenário real de BDD e Smoke Test utilizando automação de navegador.

A disseminação dos testes automatizados não para de crescer. Não é difícil de se entender a razão para isso, já que apresenta inúmeros benefícios com relação aos testes manuais. Um deles é a redução do custo, já que os testes manuais são mais lentos. Contudo, há uma vantagem do teste automatizado que supera as expectativas: é uma garantia de que o teste sempre será realizado. A forma que esses testes são armazenados, permite que a quantidade vá aumentando com o tempo, formando o que (HIBBS; JEWETT; SULLIVAN, 2009) chamam de rede. Segundo eles, essa rede passa a ser uma segurança para o desenvolvedor.

Em um cenário onde o desenvolvedor precisa acertar da primeira vez, toma-se muito cuidado ao fazer um commit para o repositório. Por outro lado, se eu sei que, assim que eu fizer um commit, será realizado um build e um conjunto de testes automatizados, me sinto mais seguro para arriscar e criar, aumentando o número de entregas que realizo. (HIBBS; JEWETT; SULLIVAN, 2009), comentam a baixa qualidade das documentações dos sistemas em geral e que os desenvolvedores estão cansados desse tipo de documento, já que, quando existem, estão desatualizados. Para não correr riscos, o desenvolvedor acaba preferindo estudar o código analisando o impacto de sua alteração, o que aumenta o tempo de execução da tarefa. (HIBBS; JEWETT; SULLIVAN, 2009) propõem os testes automatizados como substitutos da documentação do sistema: um lugar onde o comportamento que o sistema deve ter está preservado e evolui com o tempo. Essa garantia dá mais confiança ao desenvolvedor. Por isso, (HIBBS; JEWETT; SULLIVAN, 2009) diferenciam testes automáticos de

testes automatizados. O primeiro apenas se opõem a manual. Já o segundo, é uma rede de testes que definem a maneira exata de como o software deve se comportar.

Voltando ao exemplo do método *retornaElemento* contida no código 4.1. Se pensamos em um sistema de milhares de classes, desenvolvido sem testes automatizados e sem tratamento dos erros, e no mesmo sistema, desenvolvido utilizando os testes unitários automatizados, conforme exemplificado com a ferramenta JUnit, em qual cenário um desenvolvedor se sentiria seguro para realizar uma alteração? Em qual deles a curva de aprendizado seria mais rápida? Com certeza em um cenário onde sabe que há testes automatizados bem elaborados que garantem o comportamento adequado do software. Caso algo dê errado, os testes irão falhar. Esse é um dos grandes segredos de equipes produtivas, que liberam vários releases por semana. Entendemos assim o que (HIBBS; JEWETT; SULLIVAN, 2009) propõem como uma rede de documentação do software.

As corporações em geral possuem sistemas em produção sustentando sua operação. Suponhamos que um desses sistemas, que em geral possuem dezenas de milhares de linhas de código, não possua testes automatizados ao criar uma nova versão para liberar em produção. (HIBBS; JEWETT; SULLIVAN, 2009) definem este sistema como código legado. É um pouco assustadora essa definição, contudo mostra a relevância a que chegou essa automatização. Vamos supor que esta organização hipotética resolva automatizar os testes deste sistema, visando a aumentar a sua qualidade. Ao avaliar os custos desse projeto, chegaremos em um valor tão alto que, provavelmente, não será viável pagá-lo. Dessa forma, sim, o código tornou-se legado, pois já não é possível automatizar os seus testes. Fica então a experiência de que é necessário pensar com cuidado nos testes automatizados de novos projetos.

5.1 Ferramentas de Testes Automatizados

Como há muitas frentes na área de testes, por consequência, há muitas ferramentas surgindo, muitas vezes concorrendo entre elas. Na tabela 5.1 são mostradas as categorias dessas ferramentas, de acordo com sua área de aplicação. Os produtos mostrados podem ser gratuitos ou pagos.

A ferramenta Cypress merece destaque e vem crescendo em popularidade. Por ser mais moderna que sua maior concorrente, Selenium, tem um processo de utilização simplificado, apresentando-se como opção mais atrativa. Permite que o desenvolvimento de Interface de Usuário seja orientado a desenvolvimento, TDD. Está em seu roadmap a liberação de funcionalidades de automatização do navegador.

Outro destaque é a ferramenta Mockito, que pode ser utilizada tanto para criação de Mocks quanto para Testes Unitários. Em 2013, em uma análise de milhares de projetos do GitHub, ficou entre as top 10 bibliotecas Java mais utilizadas, na frente até mesmo de Spring, apesar de ter perdido para JUnit.

Por fim, vale a pena conhecer e aprofundar no uso da ferramenta Cucumber, que implementa o conceito de BDD.

5.2 Estudo de Caso: Teste Automatizado em uma Equipe Agile

Este caso é uma experiência do time de Lisa, uma Analista de Testes que conhece bastante de Agile. Faz parte de uma coletânea organizada por (GRAHAM; FEWSTER, 2012). A equipe começou sem experiência e sem nenhum teste automatizado, além do sistema possuir uma arquitetura pobre.

CATEGORIA	FERRAMENTA	CARACTERÍSTICAS
Interface Gráfica do Usuário / Automação do Navegador	Cypress, Selenium, Ranorex, Watir, Egg-Plant	Validação dos requisitos de interface do usuário. Desenvolvimento de interface orientado a TDD.
Teste Cross-browser	Saucelabs, Selenium Webdriver	Validação do comportamento correto em qualquer navegador.
Teste de Carga	JMeter, SmartBear LoadNinja	Teste de sobrecarga da aplicação.
Teste Mobile	Apium, Saucelabs, TestObject	Teste de aplicativos nativos, web responsivos ou híbridos.
Teste de API	SoapUI, Postman, Karate	Teste de API do back end através de chamadas HTTP.
Teste de Segurança	Netsparker, Accunetix	Avaliação de vulnerabilidades como injeção externa de SQL e scripts.
Teste de Usabilidade / Experiência do usuário	Optimizely, Qualaroo, Crazy Egg	Análise do comportamento do usuário. Teste A/B.
Frameworks de Testes Unitários	Mockito, JUnit, TestNG	Elaboração e estatísticas de execução de testes unitários.
Frameworks de Testes Integrados	Cucumber (BDD), Fitnesse, Spock	Elaboração e execução de testes integrados.
Cobertura de Testes	JaCoCo, Cobertura	Estatísticas de cobertura de software. Complexidade ciclomática.
Gerenciamento de Testes	qTest, PractiTest, TestLodge, Airbrake, BugZilla, Jira	Controle dos bugs. Integração com demais ferramentas. Indicadores. Workflow para gerenciamento de defeitos.

Tabela 5.1: Ferramentas de Testes

5.2 Estudo de Caso: Teste Automatizado em um Capítulo 5 Testes Automatizados

Características	Estudo de Caso
Domínio da aplicação	Serviços financeiros, Aplicação J2EE
Tamanho	226.000 Linhas de código: 95.000 legadas cobertas principalmente com GUI smoke tests. 131.000 da nova arquitetura cobertas por 4.364 testes unitários (JUnit) Em 2004, estes números eram: 128.000, 20.000 e 473, respectivamente.
Localização	EUA
Ciclo de Vida	Agile, tanto o sistema legado quanto os novos desenvolvimentos. Novas entregas a cada duas semanas (iteração ou sprint).
Pessoas no projeto	9 a 12 (De 4 a 5 Programadores, de 1 a 2 Analistas de Testes, 1 DBA, 1 Scrum-Master e 1 Gerente)
Analistas de Testes	1 a 2
Tempo decorrido	Este estudo de caso cobre principalmente o primeiro ano, mas diz a situação seis anos depois.
Período	De 2003 a 2011.
Ferramentas	Open Source
Estudo piloto realizado	Não
Avaliado Retorno do Investimento	Não medido, mas foram observados número de defeitos e velocidade de correção. (Story Points por Sprint)

Fonte: Adaptada de (GRAHAM; FEWSTER, 2012).

Tabela 5.2: Características do Projeto

A startup estava sem credibilidade, quase falindo, mas teve uma nova chance quando começou a adotar Scrum. Um ano depois, todos os testes de regressão estavam automatizados. Na tabela 5.2 é apresentado o cenário do projeto.

Problemas

- Entregar novas funcionalidades a cada duas semanas.
- Código defeituoso.
- Nenhum teste automatizado.

Objetivos

- Obter o código de melhor qualidade possível.
- Criar uma rede de segurança de testes automatizados de regressão para produzir mais rápido.
- Descobrir no ato se uma alteração específica no código introduziu comportamento inesperado, visando a estabilizar o código fonte.
- Realizar Integração Contínua com cobertura adequada de testes, objetivando um build diário.

Pontos fortes

- Comprometimento do time com a qualidade.
- Excelente comunicação.
- Atividades de teste planejadas.

Pontos fracos

- Dois dias a cada duas semanas dedicados aos testes manuais.

- Pouco tempo para realizar testes exploratórios para encontrar erros antes dos usuários.
- Feedback sobre o código obtido somente a cada duas semanas, no fim do sprint.

Os testes de regressão da equipe, além de roubar 20% do tempo, deixavam escapar alguns erros para produção. Para corrigi-los, a equipe levava cerca de 20% do tempo da iteração. Com este cenário a equipe começou a se preocupar com soluções alternativas.

Este momento é um impasse comum: apostar na automação vai liberar tempo da equipe e aumentar a qualidade do software, a ponto de pagar o investimento? Perdendo 40% do tempo com testes manuais ineficientes e correções, a equipe se viu obrigada a passar por uma curva de aprendizado, investir tempo e dinheiro, preparar ambientes e estudar frameworks que permitissem automatizar os testes.

Uma das primeiras iniciativas da equipe foi direcionar para o uso do Test-Driven Development (TDD) visando a um código de maior qualidade e a uma arquitetura que pudesse ser testada. Para piorar as coisas, a arquitetura não implementava MVC. A lógica específica de negócio, a camada de apresentação e o acesso ao banco estavam misturados, impossibilitando isolar qualquer componente para testar. Neste momento, percebeu-se que utilizar TDD trata-se praticamente de reescrever o código, de refazer a arquitetura do sistema. Um trabalho bastante ousado.

Abaixo, podemos ver a Pirâmide de Testes adotada. Este conceito faz parte da metodologia Agile. Os testes unitários estão na base, em maior quantidade, porque são fáceis de escrever e rápidos para serem executados, trazendo um melhor retorno do investimento. Em quantidade intermediária, no meio da pirâmide, estão os testes de aceitação que avaliam a lógica do negócio. No topo da pirâmide estão os testes de interface do usuário. Em contraste com os testes unitários, são caros e lentos para executar, por isso estão em menor número. Por último, os testes manuais, conhecidos como Smoke Tests ou BVT. Estes testes são realizados sempre que um build da aplicação é gerado.

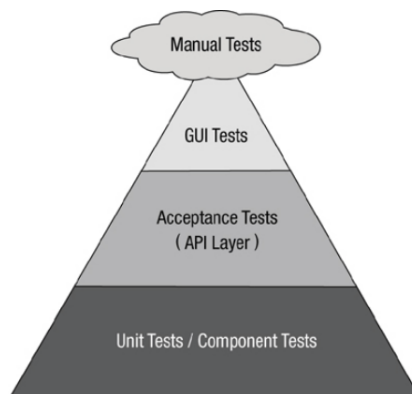


Figura 5.1: Pirâmide de Testes

Fonte: Adaptada de (GRAHAM; FEWSTER, 2012).

Integração Contínua

Nesse estágio, a aplicação já possui uma nova arquitetura. Algo que está sendo fonte de problemas são os builds. Como garantir ou diminuir as chances de um commit para o repositório feito pelo desenvolvedor não gerar um defeito em algo que estava funcionando corretamente? Como evitar que um build seja realizado sem que um desenvolvedor precise parar para fazê-lo? Lisa relata um problema frequente pelo qual passa: esperar que um build seja realizado para que possa testar uma funcionalidade que já está pronta. Aqui a equipe decide implementar a integração contínua: após o

commit, os testes devem ser executados e gerar um build pronto para ser distribuído.

É interessante como Lisa aborda a evolução dos desenvolvedores com o tempo. Muitas vezes temos expectativas de que as pessoas produzam com qualidade desde o princípio, mas é natural que precisem de tempo para melhorar a performance. Lisa relata que os desenvolvedores precisaram de alguns meses para aprender a automatizar testes unitários e escrever os casos de teste antes de desenvolver o código.

No momento de escolher ferramentas para automatizar Smoke Tests de interface de usuário para o código legado, ou seja, código sem testes, os programadores Java se opuseram a utilizar ferramentas de outras linguagens. Lisa usa o termo: “eles não querem”. Essa atitude é bastante frequente no mundo dos desenvolvedores. Soa bastante a imaturidade, confesso que isso me assusta um pouco. Contudo, existiria uma curva de aprendizado mais longa, caso a equipe precisasse estudar a nova linguagem, além da nova ferramenta. A ferramenta precisaria justificar esse maior esforço.

No processo de automatização desses Smoke Tests para o código legado, a equipe priorizou algumas funcionalidades chave de cada papel de usuário do sistema. Essa estratégia é inteligente, pois traz resultados rápidos.

Num primeiro momento, o build de integração contínua executava apenas alguns casos de teste e alguns GUI Smoke Tests. Conforme os testes foram crescendo, Lisa separou o build de Smoke Tests para executar. Caso contrário, o feedback dos testes unitários seria muito lento.

Após estudar boas práticas, foi decidido que o build de testes unitários precisaria ocorrer dentro de dez minutos. Isso auxilia a reescrever algum teste, caso se perceba que pode ser melhorado. Logo no início os desenvolvedores começaram a fazer com que os testes unitários acessassem o banco de dados. Eles aprenderam como fazer stubs e mocks nessa conexão para manter a rapidez de execução dos testes sem perder cobertura. Depois de oito meses a equipe possuía uma biblioteca com 100 testes unitários JUnit e Smoke Tests para todas as funcionalidades críticas da aplicação. Essa estratégia permitiu que cada nova funcionalidade fosse testada nos dois níveis da pirâmide escolhidos até aqui dentro da mesma iteração. Quando os programadores começaram a desenvolver bons testes TDD, passou-se a focar na camada de testes de aceitação, nível intermediário da pirâmide que trabalha as regras do negócio.

O desenvolvimento de novas User Stories (funcionalidades) da camada de negócio foi orientado a TDD. Como se trata de um software de serviços financeiros, a complexidade é alta. Consequentemente, a estratégia de teste também é complexa. A equipe selecionou a ferramenta FitNesse nessa camada. Ela trabalha com tabelas de resultados esperados para cada entrada. O funcionamento simula ambiente de produção criando as entradas em tempo de execução, chamando o código, obtendo o retorno e comparando com a tabela especificada inicialmente. Para novos testes, basta ir adicionando registros nessa tabela. Bastante prático, deixando o trabalho na mão do criador dos testes, que pode usar sua capacidade crítica para encontrar bugs. Contudo, como o desenvolvedor precisa implementar o dispositivo FitNesse que chama seu código, a comunicação entre desenvolvedores, analistas de testes e equipe do negócio aumentou muito, pois era preciso definir muito bem essas regras. Lisa considerou isso como uma grande vantagem do cenário escolhido para testes em uma camada tão crítica.

Com o tempo, a equipe percebeu que havia muitos testes no conjunto automatizado de testes de regressão. Tornou-se custoso manter e demorado para executar. Lisa compartilha uma experiência importante: identificar e selecionar os melhores testes, sem diminuir a cobertura do software. Os testes FitNesse do projeto levavam entre 60 e 90 minutos para executar, em comparação com os 8 minutos dos testes JUnit. Inicialmente, foram adicionados no processo noturno, juntos com os Smoke Tests. Mais tarde, pelo problema de ter que esperar o dia seguinte para verificar o resultado de um

check-in de código, a equipe investiu em hardware para executar os testes em Integração Contínua, demorando 90 minutos para finalizar. Assim o feedback era mais rápido, consequentemente a produtividade também. Lisa relata que é necessário haver um equilíbrio entre cobertura do teste e velocidade da execução para a iteração correr bem. Muitas vezes precisaram de criatividade para os testes executarem mais rápido, como executar em paralelo ou usar máquinas virtuais.

O time de Lisa procurou fazer Marketing dos testes automatizados para seu cliente. A cada iteração, enviava um relatório marcando os dias com a cor verde, caso os testes de regressão ocorressem sem erros pelo menos uma vez. Também enviava o número de testes realizados para cada camada. Se alguma dessas métricas caísse, o cliente entrava em contato para obter informações, mostrando que a cultura do teste automatizado se estabeleceu.

5.3 Estudo de Caso: Google GWS

Este caso está relatado por (KIM, GENE, 2017), entusiasta de DevOps, autor de diversos livros sobre o assunto. Descreve os problemas de deploy no servidor Google Web Server (GWS), o próprio google.com, em 2005, antes da Companhia se tornar uma das culturas mais admiradas de testes automatizados.

Seguindo boas práticas de desenvolvimento, novas funcionalidades de sistema são desenvolvidas e testadas em ambientes que simulam produção, em seguida sendo enviadas para o repositório de software. Contudo, diversos problemas podem ocorrer quando os testes são realizados separadamente, depois que o desenvolvimento foi concluído. Se os testes ocorrem apenas algumas vezes por ano, os desenvolvedores vão demorar muito para saber que seus commits de código causaram defeito. A primeira perda é não poder aprender com os erros, não podendo refletir e adotar boas práticas. Perde-se o link de causa e efeito, sendo necessário realizar um trabalho de arqueologia para encontrar o que causou o problema. Sabemos que, depois de alguns meses, um desenvolvedor precisa estudar novamente seu código, principalmente se for de maior complexidade.

Em correspondência pessoal com Gene Kim, Gary Gruver observa que na ausência de testes automatizados, quanto mais código é escrito, mais tempo e mais dinheiro é despendido para testar: uma atividade não escalável.

Nem sempre o Google foi uma companhia exemplo de cultura de testes automatizados. Em 2005, quando Mike Bland começou a trabalhar na organização, fazer deploy para Google.com era sempre problemático, principalmente para o time GWS. Conforme explicação do próprio Bland:

“Por volta de 2005, o time GWS chegou em um ponto onde era muito difícil realizar mudanças no web server, uma aplicação C++ que gerenciava todas as requisições para a Home Page do Google e muitas outras páginas. Apesar da importância e proeminência da corporação, estar no GWS não era atribuição glamourosa. Frequentemente era o lugar para onde ia todo código relacionado a funcionalidades de busca, completamente diferentes uns dos outros. Tinham problemas frequentes com builds e testes demorados, código colocado em produção sem testes e equipes fazendo check-in de código em larga escala, causando conflito uns com os outros”.

Essa situação tinha vastas consequências: os resultados da busca poderiam conter erros ou se tornar lentos, afetando milhares de consultas. Como usuários sabemos como o Google lida com a relação de confiança. Pela qualidade de suas aplicações, é sabido que isso é uma preocupação primordial. Em 2005, devido a esse cenário, isso começou a ser quebrado, acarretando também em perda de receita.

Não é difícil imaginar as preocupações do desenvolvedor ao fazer deploy. O medo começou a predominar, paralisando a todos. Tanto os novos, que não sabiam como as coisas funcionavam,

quanto os colaboradores antigos, que sabiam muito bem. Bland fazia parte do grupo disposto a resolver este problema.

“Foi dada uma ordem: nenhuma mudança seria aceita no GWS sem testes automatizados. Foi criado um build contínuo, seguido religiosamente. A cobertura de teste começou a ser monitorada e foi assegurado que esse indicador aumentasse com o tempo. Foram escritos manuais de teste e se insistiu para que colaboradores, dentro e fora do time, os seguissem”, afirma Bland.

Notamos aqui uma atitude firme da companhia, necessária, de acordo com a situação. Foi apresentada a solução para o problema, no caso a automatização, e como medir resultados. Agora o build e consequentemente os testes estão cronologicamente próximos ao desenvolvimento, possibilitando que os programadores aprendam com os erros.

Conforme relato de Bland, o cenário mudou: “rapidamente o GWS se tornou um dos times mais produtivos da companhia, integrando um número grande de alterações dos diferentes times, semanalmente, mantendo uma agenda de release rápida. Novos membros passaram a contribuir para o sistema mais rapidamente, graças à cobertura de teste e saúde do código”. Verificou-se mais uma vez que a rede composta pelos testes automatizados descrita por (HIBBS; JEWETT; SULLIVAN, 2009) funciona realmente como uma documentação do sistema, servindo de orientação ao desenvolvedor. É razoável que neste cenário ele se sinta mais confiante para fazer check-in de seu código. Com o objetivo de expandir as práticas para todas as equipes da organização, foi criado o Testing Grouping: um grupo informal de engenheiros que, durante os próximos cinco anos, auxiliaram a replicar a cultura de testes automatizados.

Atualmente, quando qualquer desenvolvedor realiza um commit, ele é automaticamente submetido a um conjunto de centenas de milhares de testes automatizados. Se o código passar, é realizado um merge para o trunk (versão principal), pronto a ser enviado para produção.

Caso o código não passe, o pipeline do deploy é interrompido. Se for uma biblioteca de uso global, centenas de engenheiros entraram em contato, informando que precisam de auxílio na correção. Essa característica do novo ambiente também assusta o desenvolvedor que precisa reverter a alteração ou realizar a correção imediatamente. Contudo, na cultura do Google, parte-se do princípio que todos querem acertar, gerando uma cadeia de confiança, que todos farão de tudo para manter o pipeline do deploy acontecendo normalmente.

Com certeza, trata-se de um caso de sucesso, pois a companhia tornou-se uma das mais produtivas organizações de tecnologia do mundo. Os números são impressionantes: em 2013, a automatização de testes e integração contínua no Google permitiram que mais de 4000 times pequenos trabalhassem juntos e permanecessem produtivos, desenvolvendo simultaneamente, integrando, testando e realizando deploy de código para produção. Todo o código está em um único repositório compartilhado, composto por bilhões de arquivos, todos sendo continuamente integrados, com 50% do código sendo alterado por mês. Outras estatísticas impressionantes de performance:

- Commits diários: 40.000
- Builds em dias úteis: 50.000
- Builds em finais de semana: 90.000
- Conjuntos de casos de testes automatizados: 120.000
- Casos de teste executados por dia: 75.000.000

(KIM, GENE, 2017) cita que mais de 100 engenheiros foram alocados em áreas relacionadas, visando a aumentar a produtividade do desenvolvedor.

Pontos de Atenção

Ao se deparar com resultados tão satisfatórios dos testes automatizados, podemos ficar pensando: mas será que tudo sempre corre tão bem? Quais são as ameaças existentes desse cenário que podem afetar a qualidade do software entregue, mesmo com um grande número de testes automatizados sendo executados? Isso fica evidenciado quando se tem um processo bem definido de testes automatizados, mas o número de defeitos encontrados por usuários finais não diminui.

Em seu artigo, (GHAHRAI, AMIR, 2019) faz o seguinte questionamento: estamos entregando software de maior qualidade com mais testes automatizados? A resposta é não. Segundo ele, podemos chegar em um cenário onde muitos casos de teste ruins são executados sem encontrar os erros presentes no código. O que ocorre é que cada vez mais erros passam a ser encontrados nos Testes de Aceitação do Usuário. Uma situação bastante frustrante para a equipe de QA (*Quality Assurance*).

(GHAHRAI, AMIR, 2019) enfatiza a necessidade de se preparar bem os Analistas de Testes, fomentar sua criticidade. Não podemos deixar que esses Analistas estejam preocupados somente em ver o pipeline de testes executar sem erros, ficando todos na cor verde, simplesmente. Isso pode ser um grande engano. Reforça a necessidade dos Testes Exploratórios onde os chamados Testers começam a explorar cenários variados que podem fazer os defeitos aparecer. Quem já viu uma pessoa com essa capacidade trabalhando, fica encantado, pois é uma forma de pensar indireta, sem seguir o caminho padrão de uso do software. É um raciocínio pautado por frases do tipo: e se o usuário começar por aqui, em vez de seguir o caminho padrão? E se o usuário preencher texto em vez de número? Percebemos que muitas dessas situações podem se repetir em projetos de software diferentes, o que leva o analista a se acostumar com cenários de teste parecidos. Contudo, é uma habilidade valiosa saber encontrar comportamentos inesperados nos diferentes softwares sendo testados.

Ainda nessa questão, (GHAHRAI, AMIR, 2019) descreve uma situação frequente, principalmente em desenvolvimento Agile. É comum um Analista de Testes automatizar o Critério de Aceitação de uma User Story, por exemplo, e, em seguida, tendo um pobre conhecimento de programação, criar um caso de teste simples para poder terminar rapidamente a tarefa e ver o teste executar sem erros. Aqui podemos ter uma das raízes do problema. Dessa forma, tende-se a esquecer o cenário real e a dimensão do teste que é necessário para testar com qualidade o software.

O trabalho de (GHAHRAI, AMIR, 2019) traz cinco erros cometidos em testes automatizados. São eles:

- *Expectativas erradas com os testes automatizados.* Seria incorreto almejar a descoberta de diversos defeitos de software nesta fase, já que poderia colocar em questão a capacidade técnica dos desenvolvedores.
- *Automatizar testes na camada errada, com as ferramentas erradas e no momento errado.* Aplicações modernas estão sendo desenvolvidas separadas em Front e Back End. Dessa forma, o teste e a ferramenta devem ser adequados à camada sendo testada. O teste automatizado deve ser simultâneo ao desenvolvimento, já que mais erros são encontrados nesta fase do que durante os testes de regressão.
- *Automatizar testes que não trazem resultado.* Considerando os custos envolvidos, deve haver um equilíbrio entre a quantidade de testes e sua qualidade.
- *Negligenciar áreas importantes.* A Integração e Entrega Contínuas, presentes na metodologia DevOps de Delivery Pipeline (Pipeline de Entrega), não costumam ser testados. O autor chama atenção para o fato de muitos erros não serem do software em si, mas do processo de build e da entrega automatizada. Isso pode ocorrer com um arquivo de properties que aponta para um

ambiente de homologação ir para produção, por exemplo. Um problema como esse poderia fazer com que o sistema de produção apontasse para o banco de homologação.

- *Esquecimento de cenários de teste fundamentais.* Um erro chega à produção porque ninguém previu um cenário de teste que o revelaria.

5.4 Provas de Conceito de Testes Automatizados

5.4.1 JUnit utilizando TDD

Neste POC (*Proof of Concept*), iremos utilizar Desenvolvimento Orientado a Testes, TDD, para melhorar a classe Vetores. A versão inicial da classe está bastante vulnerável a erros, conforme podemos observar no código 5.1.

```
1 public class Vetores {
2
3     String retornaElemento(String[] a, int n) {
4
5         return a[n];
6
7     }
8
9 }
```

Código Fonte 5.1: A classe Vetores e a função *retornaElemento*

De acordo com a estratégia escolhida, primeiro são escritos os testes. Num primeiro momento, pode parecer um pouco estranho, mas basta utilizar a técnica exploratória para perceber o que pode ocasionar erros na classe. Conforme visto anteriormente, precisam ser tratadas referências nulas, array vazio etc. Depois dessa exploração, foram escritos os testes abaixo, antes de alterar o método *retornaElemento*.

A classe VetoresTest representada no código 5.2 utiliza a ferramenta JUnit 5. Possui cinco testes que documentam o comportamento esperado para o método *retornaElemento*. O primeiro teste executado verifica se a função sempre devolve o elemento do índice pedido pelo Array. Se não devolver o valor esperado, para todos os índices do vetor, os testes finalizam com erros. O teste do método *deveLancarExcecaoParaArrayNulo* trata um problema crítico em linguagens orientadas a objetos: uma referência nula, ou seja, que não aponta para objeto algum. Veja que na assinatura do método *retornaElemento* é informado que ele lança uma exceção do tipo *Exception*. Isso obriga aos que chamam o método a tratar este problema: se o vetor estiver nulo, não chame este método, ou trate a exceção que ele vai devolver. O teste do método seguinte, tem um papel semelhante, mas agora para o caso de um array vazio. Já os dois últimos testes, documentam o comportamento do método *retornaElemento* quando o vetor não possui o índice informado. A nomenclatura utilizada para a classe é uma boa prática que facilita a rápida localização da classe sob teste. Sempre se utiliza o nome da classe finalizado pela palavra *Test*. Neste caso: *VetoresTest*. O nome do método pode ser iniciado com a palavra *deve* (*should*), seguido do comportamento desejado para aquele caso de teste.

```
1 package temp;
2
```

```
3 import static org.junit.Assert.assertTrue;
4
5 import org.junit.jupiter.api.Assertions;
6 import org.junit.jupiter.api.Test;
7 import org.junit.platform.runner.JUnitPlatform;
8 import org.junit.runner.RunWith;
9
10 @RunWith(JUnitPlatform.class)
11 public class VetoresTest {
12
13     public String[] vetor = { "Texto 1",
14                               "Texto 2" };
15     public Vetores v = new Vetores();
16
17     @Test
18     void deveRetornarElementoDeAcordoComIndice() {
19         var i = 0;
20         for (var elemento : vetor) {
21             Assertions.assertEquals(elemento,
22                                     v.retornaElemento(vetor, i));
23             i++;
24         }
25     }
26
27     @Test
28     void deveLancarExcecaoParaArrayNulo() {
29         Exception e = Assertions.assertThrows(
30             RuntimeException.class,
31             () -> v.retornaElemento(null, 0));
32         assertTrue(e.getMessage()
33                     .contentEquals("Array nulo"));
34     }
35
36     @Test
37     void deveLancarExcecaoParaArrayVazio() {
38         String[] vetor = {};
39         Exception e = Assertions.assertThrows(
40             RuntimeException.class,
41             () -> v.retornaElemento(vetor,
42                                     0));
43         assertTrue(e.getMessage()
44                     .contentEquals("Array vazio"));
45     }
46
47     @Test
48     void deveLancarExcecaoParaIndiceNegativo() {
```

```
49         Exception e = Assertions.assertThrows(  
50             RuntimeException.class,  
51             () -> v.retornaElemento(vetor,  
52                 -1));  
53         assertTrue(e.getMessage().contentEquals(  
54             "Fora do dominio"));  
55     }  
56  
57     @Test  
58     void deveLancarExcecaoParaIndiceMaiorQueArray() {  
59         Exception e = Assertions.assertThrows(  
60             RuntimeException.class,  
61             () -> v.retornaElemento(vetor,  
62                 555555));  
63         assertTrue(e.getMessage().contentEquals(  
64             "Fora do dominio"));  
65     }  
66  
67 }
```

Código Fonte 5.2: A classe VetoresTest

Como era esperado, ao rodar os testes automatizados, ocorrem as falhas, conforme 5.2. Perceba que apenas o teste do chamado “caminho feliz” teve sucesso. Também podemos observar que a ferramenta JUnit diferencia uma falha de asserção (letra X azul) de um erro de programação (letra X vermelha).

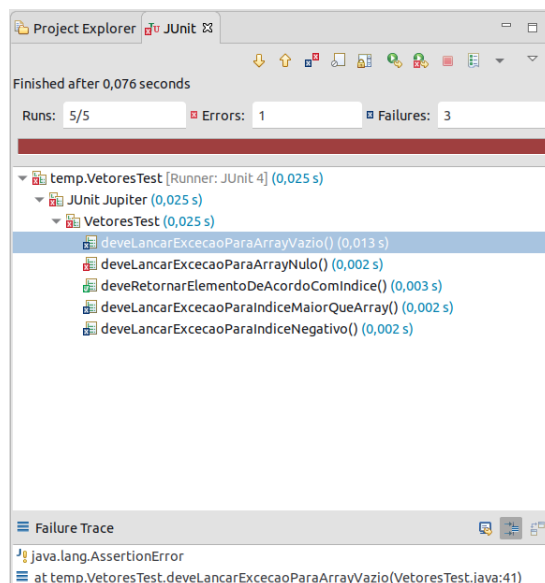


Figura 5.2: Testes JUnit encontram erros presentes no código

Prosseguindo no uso de TDD, chegou o momento de realizar o desenvolvimento, escrevendo

o código que faz com que o método se comporte da maneira documentada nos testes escritos. No código 5.3, vemos a versão melhorada da classe Vetores. Agora, o método retornaElemento está protegido contra erros bastante comuns de manipulação de vetores.

```
1 public class Vetores {  
2  
3     public String retornaElemento(String[] a, int n)  
4         throws Exception {  
5  
6         if (a == null) {  
7             throw new RuntimeException("Array nulo");  
8         }  
9  
10        if (a.length == 0) {  
11            throw new RuntimeException("Array vazio");  
12        }  
13  
14        if (n < 0 || n >= a.length) {  
15            throw new RuntimeException("Fora do dominio");  
16        }  
17  
18        return a[n];  
19    }  
20 }  
21  
22 }
```

Código Fonte 5.3: A classe Vetores após correção de defeitos

Atingindo o objetivo planejado inicialmente, depois da alteração da classe Vetores, os testes ocorrem com sucesso, como pode ser observado na figura 5.3.

Vale ressaltar que a criação dos testes unitários levou o código à uma melhor versão. Conforme os testes foram sendo criados, foi sendo necessário estruturar o código utilizando-se de boas práticas de tratamento de exceções. E ao final disso, ficou documentado o comportamento que o software deve ter. No futuro, se alguma alteração for realizada no método, durante a execução dos testes será validado se o comportamento esperado foi alterado ou não, requerendo uma ação do desenvolvedor em caso positivo. Veja que esse é o cenário ideal para o desenvolvedor: o teste funcionando como documentação, dando-lhe garantias e segurança para alterar o código com maior frequência, permitindo que ele produza mais.

5.4.2 BDD e Smoke Test utilizando Cucumber e Selenium

Neste POC vamos conhecer o comportamento da ferramenta Cucumber, implementando uma User Story cujo cenário é realizar login em um sistema de condomínio. O processo partirá da especificação e terminará em um teste real, com automatização do navegador utilizando a ferramenta Selenium.

Na figura 5.4 é apresentada a estrutura do projeto Java, na IDE Eclipse, e também a User Story no arquivo *Condominio.feature*. Perceba que o texto está formatado, com destaque de palavras

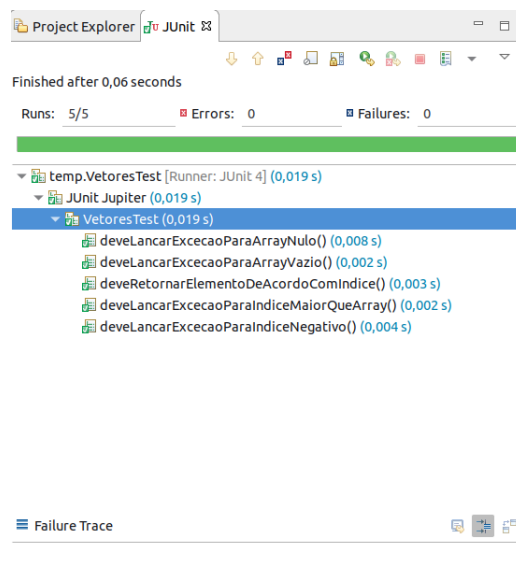


Figura 5.3: Testes JUnit executados com sucesso

reservadas. Isso é possível ao utilizar o plugin Cucumber para Eclipse, que reconhece a extensão *.feature*.

O código 5.4 da classe TestRunner tem o papel de utilizar JUnit para disparar os testes da ferramenta Cucumber. Basicamente, quando executada, irá procurar especificações para executar na pasta *Feature* e o código correspondente de cada linha da User Story na pasta *seleniumgluecode*.

```

1 package runner;
2
3 import org.junit.runner.RunWith;
4
5 import cucumber.api.CucumberOptions;
6 import cucumber.api.junit.Cucumber;
7
8 @RunWith(Cucumber.class)
9 @CucumberOptions(features = "Features", glue = {"seleniumgluecode"})
10 public class TestRunner {
11
12 }

```

Código Fonte 5.4: A classe TestRunner

Ao executar a classe TestRunner, caso não exista o código correspondente na pasta *seleniumgluecode*, a ferramenta Cucumber transforma a especificação do arquivo *Condominio.feature* em código fonte, conforme exemplo da figura 5.5.

Dessa forma, é possível copiar o código gerado e elaborar a classe *Condominio.java*. Essa classe é uma ligação entre a especificação e o teste que realiza a automatização do navegador. É chamada de "cola", uma ligação entre uma linha da especificação e um método para teste. No código 5.5 é

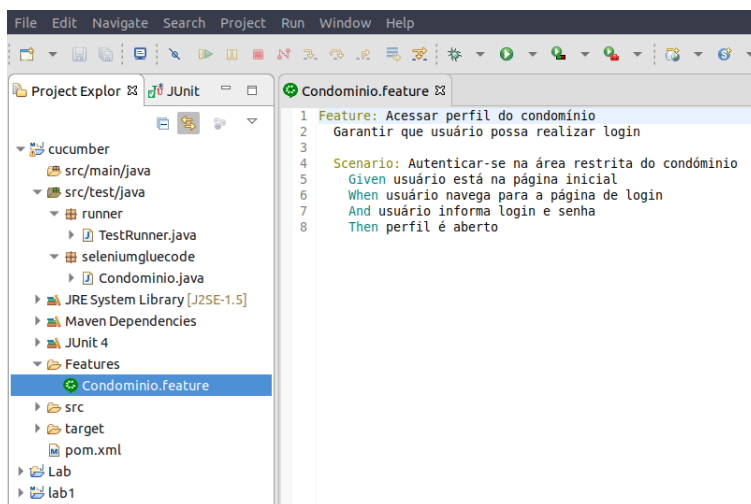


Figura 5.4: User Story utilizando linguagem Gherkin na ferramenta Cucumber

descrita a classe já com o código Selenium de automatização do *browser*.

```

1 package seleniumgluecode;
2 import org.junit.Assert;
3 import org.openqa.selenium.By;
4 import org.openqa.selenium.WebDriver;
5 import org.openqa.selenium.firefox.FirefoxDriver;
6 import cucumber.api.java.en.Given;
7 import cucumber.api.java.en.Then;
8 import cucumber.api.java.en.When;
9
10 public class Condominio {
11
12     WebDriver driver;
13
14     @Given("^usuário está na página inicial\$")
15     public void usuário_está_na_página_inicial() throws Throwable {
16         System.setProperty("webdriver.gecko.driver",
17             "/home/nobre/dev/geckodriver-v0.24.0-linux64/geckodriver");
18         driver = new FirefoxDriver();
19         driver.manage().window().maximize();
20         driver.get("https://www.villageparana.com.br");
21     }
22     @When("^usuário navega para a página de login\$")
23     public void usuário_navega_para_a_página_de_login()
24         throws Throwable {
25         driver.findElement(By.id("lnkRestritoSite")).click();
26     }

```

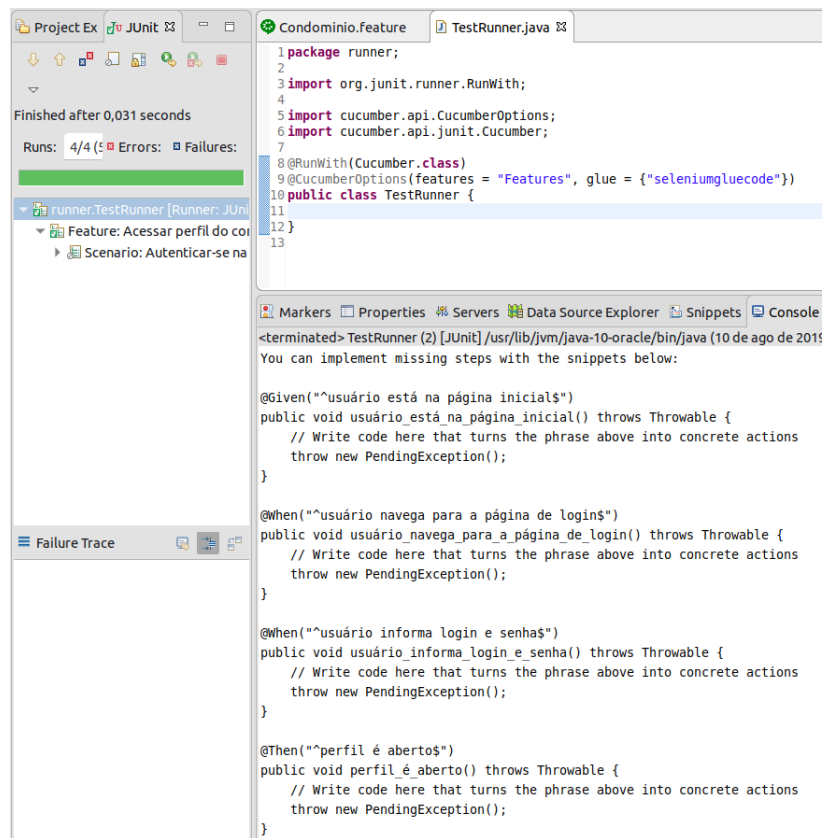


Figura 5.5: Código gerado pela ferramenta Cucumber

```

27 @When("^usuário informa login e senha$")
28 public void usuário_informa_login_e_senha() throws Throwable {
29     driver.findElement(By
30         .xpath("//*[@id=\"ContentPlaceholder3_ucLogin1_txtUsuario\"]"))
31         .sendKeys("tiago.nobre@gmail.com");
32     driver.findElement(
33         By.xpath("//*[@id=\"ContentPlaceholder3_ucLogin1_txtSenha\"]"))
34         .sendKeys("Cucumber123");
35     driver.findElement(
36         By.xpath("//*[@id=\"ContentPlaceholder3_ucLogin1_lnkAcesso\"]"))
37         .click();
38 }
39 @Then("^perfil é aberto$")
40 public void perfil_é_aberto() throws Throwable {
41     String urlEsperada = "https://app.vidadesindico.com.br/app/cond.aspx";
42     String urlObtida = driver.getCurrentUrl();
43     Assert.assertEquals(urlEsperada, urlObtida);
44     driver.quit();
45 }

```

Código Fonte 5.5: A classe Condominio

No código 5.5 pode ser vista a classe `Condominio` já com os métodos implementados. No método `usuário_está_na_página_inicial` é informado onde está na máquina local o driver do navegador a ser automatizado. No caso, escolhemos o Firefox. O driver abre o navegador, maximiza a janela e direciona para o endereço `https://www.villageparana.com.br`, conforme User Story. Ao final desse método, o estado do navegador se encontra conforme a figura 5.6. Perceba que aparece um robô na barra de endereços, sinalizando que o *browser* está automatizado.

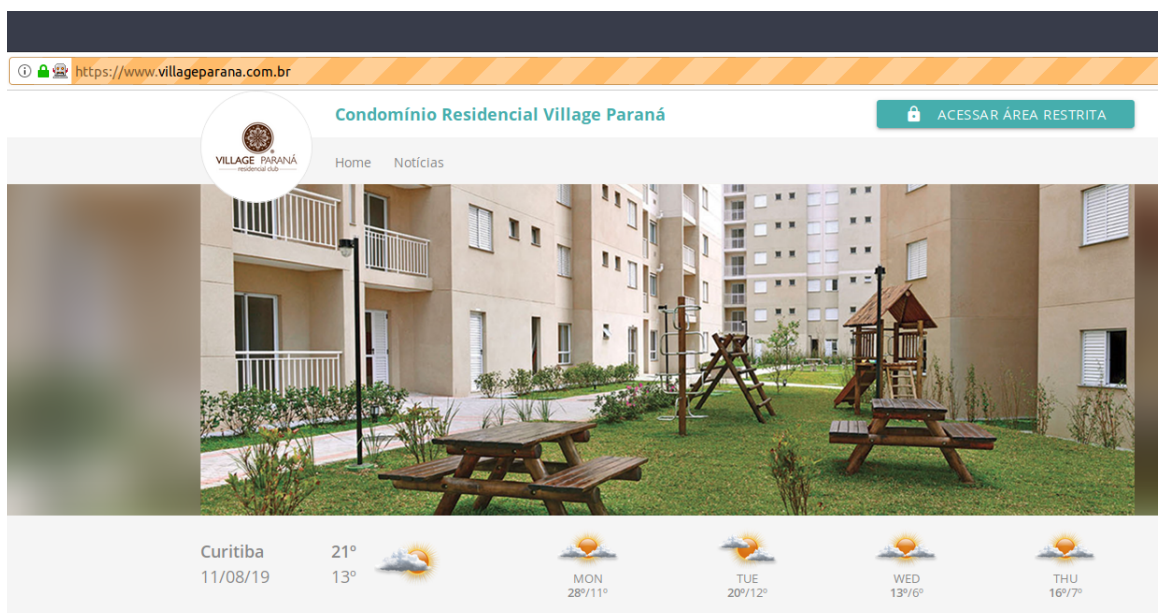


Figura 5.6: Automação do navegador com Selenium

No método `usuário_navega_para_a_página_de_login`, é simulado um clique no botão **ACESSAR ÁREA RESTRITA**, direcionando para a página ilustrada na figura 5.7. Analisando o código 5.5, é possível verificar que o botão foi capturado na página pelo seu *id*: `lnkRestritoSite`. Perceba que o driver da ferramenta é capaz de encontrar componentes de tela e simular todos os eventos que o usuário realiza. Como alguns componentes podem ser ocultados quando a tela é minimizada, o teste começa com a maximização da tela, para evitar que algum recurso não seja encontrado durante a execução do teste. Os elementos que compõem a página podem ser capturados por *id*, nome, classe *css* e *xpath*. Esse último é um identificador exclusivo fornecido pelas ferramentas de desenvolvimento do navegador, podendo ser absoluto ou relativo, com sintaxe abreviada.

Em seguida, no método `usuário_informa_login_e_senha`, são capturados os campos de texto *Usuário* e *Senha* e também o botão **ACESSAR COM SEGURANÇA**. Os campos são preenchidos com seus respectivos valores e, ao final, simula-se um clique no botão, com o objetivo de realizar o login.

Se o login for realizado com sucesso, o endereço do navegador deverá ser, obrigatoriamente, o endereço a seguir: `https://app.vidadesindico.com.br/app/cond.aspx`. No método `perfil_é_aberto` esse endereço é conferido com uma asserção JUnit. Se o endereço esperado for diferente do obtido, entende-se que o Smoke Test falhou. Esse caso está representado na figura 5.8.

https://www.villageparana.com.br/area_restrita.aspx?id=Rm0yaSa6f5OuxOdfgn5a7A==&p1=da523a16-df9a-4ed2-ae1f-be341ae62809

Condomínio Residencial Village Paraná

Home Notícias

Acesse a área restrita de seu condomínio!

Já é cadastrado?

Utilize seu usuário e senha já cadastrado para acessar a área restrita.

Usuário

tiago.nobre@gmail.com

Senha

ACESSAR COM SEGURANÇA

[Esqueceu sua senha?](#)

Primeiro acesso

Se você ainda não tem seus dados de acesso, solicite agora mesmo!

Clique aqui para solicitar acesso!

Figura 5.7: Preenchimento automático dos campos Usuário e Senha

Quando os endereços são iguais, entende-se que o login ocorreu com sucesso, conforme representado na figura 5.8.

5.5 Conclusões

Neste capítulo, pudemos ter uma visão geral das ferramentas e de suas categorias, podendo ter um caminho sobre o que adotar em cada caso. Além disso, abordamos como os testes automatizados podem se tornar documentação viva do sistema, definindo uma rede de segurança, dando mais autonomia e proatividade ao desenvolvedor. Também pudemos compreender a relação entre testes automatizados e entrega de novas funcionalidades. Ao final, foram apresentadas duas provas de conceito: um cenário real de desenvolvimento TDD e outro de BDD validando a funcionalidade com um Smoke Test.

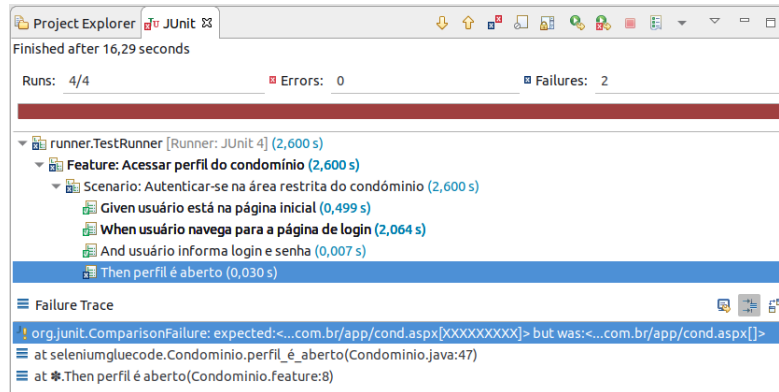


Figura 5.8: Finalização do Smoke Test com falha

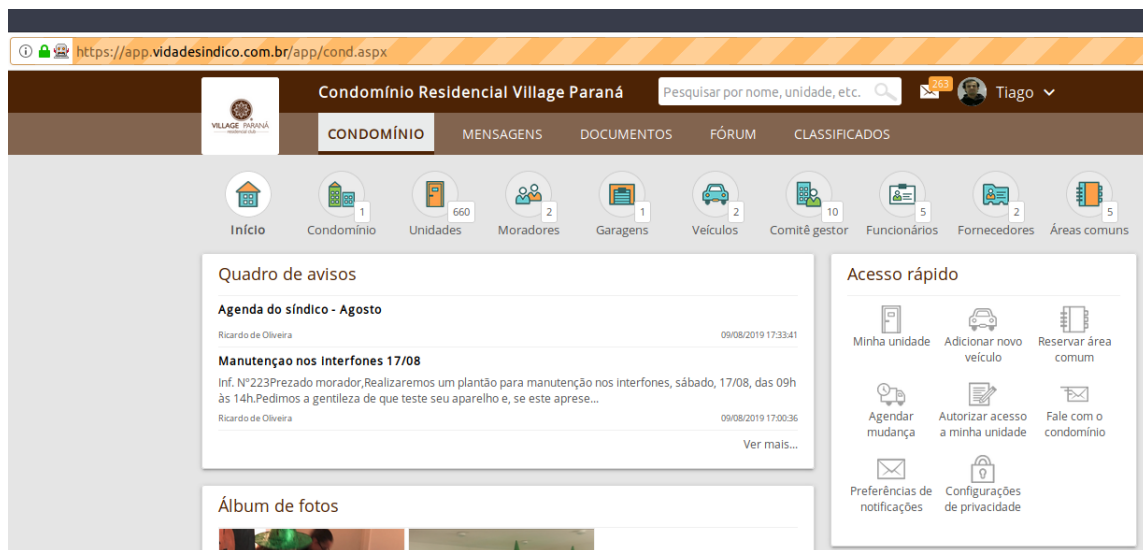


Figura 5.9: Finalização do Smoke Test, realizando login com sucesso



Referências

4LINUX. *O que é DevOps*. Acessado: 11/02/2019. Disponível em: <<https://www.4linux.com.br/o-que-e-devops>>. Acesso em: 14 fev. 2019. Citado na página 12.

AGILE ALLIANCE. *Manifesto para Desenvolvimento Ágil de Software*. 2001. Disponível em: <<https://agilemanifesto.org/iso/ptbr/manifesto.html>>. Acesso em: 14 fev. 2019. Citado na página 9.

AMIDO. *A Case Study of DevOps at Netflix*. 2019. Disponível em: <<https://amido.com/blog/a-case-study-of-devops-at-netflix>>. Acesso em: 3 ago. 2019. Citado na página 39.

APACHE SOFTWARE FOUNDATION. *Apache Ant 1.10.5 Manual*. 2019. Disponível em: <<https://ant.apache.org/manual/>>. Acesso em: 28 abr. 2019. Citado na página 25.

_____. *Introduction to the Standard Directory Layout*. Apache Software Foundation. 2019. Disponível em: <<https://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html>>. Acesso em: 28 abr. 2019. Citado nas páginas 25, 26.

_____. *Maven in 5 Minutes*. Apache Software Foundation. 2019. Disponível em: <<https://maven.apache.org/guides/getting-started/maven-in-five-minutes.html>>. Acesso em: 28 abr. 2019. Citado na página 26.

ATLASSIAN. *O que é DevOps?* Disponível em: <<https://br.atlassian.com/devops>>. Acesso em: 14 fev. 2019. Citado nas páginas 9–11.

ATLASSIAN, INC. *DevOps Maturity Model*. 2019. Disponível em: <<https://www.atlassian.com/devops/maturity-model>>. Acesso em: 15 jul. 2019. Citado na página 31.

AWS DEVOPS. *O que é DevOps?* Amazon. Disponível em: <<https://aws.amazon.com/pt/devops/what-is-devops/>>. Acesso em: 14 fev. 2019. Citado nas páginas 7, 10–13.

BAELDUNG. *Ant vs Maven vs Gradle*. 2018. Disponível em: <<https://www.baeldung.com/ant-maven-gradle>>. Acesso em: 27 abr. 2019. Citado nas páginas 22–25, 27.

BROWN, A.; DIBERNARDO, M. *500 Lines Or Less: Experienced Programmers Solve Interesting Problems.*: 2016. (Architecture of open source applications series). ISBN: 9781329871274. Disponível em: <<http://aosabook.org/en/500L/a-continuous-integration-system.html>>. Citado na página 29.

BROWN, C. Titus; CANINO-KONING, Rosangela. In: BROWN, Amy; WILSON, Greg. *The Architecture of Open Source Applications, Volume I.*: 2011. Continuous Integration. ISBN: 9781257638017. Disponível em: <<http://aosabook.org/en/integration.html>>. Citado nas páginas 17, 28.

CHACON, Scott; STRAUB, Ben. *Pro Git.*: Apress, 2014. (The expert's voice). ISBN: 9781484200766. Disponível em: <<https://git-scm.com/book/en/v2>>. Citado nas páginas 20, 21.

DAS, Malini. In: BROWN, A.; DIBERNARDO, M. *500 Lines Or Less: Experienced Programmers Solve Interesting Problems.*: 2016. A Continuous Integration System. (Architecture of open source applications series). ISBN: 9781329871274. Disponível em: <<http://aosabook.org/en/500L/a-continuous-integration-system.html>>. Citado na página 28.

DAVIS, Jennifer; DANIELS, Ryn. *Effective DevOps: Building a Culture of Collaboration, Affinity, and Tooling at Scale.*: O'Reilly Media, 2016. ISBN: 9781491926420. Disponível em: <<https://books.google.com.br/books?id=n01FDAAAQBAJ>>. Citado nas páginas 7, 8.

DEBOIS, Patrick. Agile Infrastructure and Operations: How Infra-gile are You? In: *Agile 2008 Conference*. Toronto, ON, Canada: IEEE, ago. 2008. páginas 202–207. DOI: 10.1109/Agile.2008.42. Citado na página 8.

DUVALL, P.M.; MATYAS, S.; GLOVER, A. *Continuous Integration: Improving Software Quality and Reducing Risk.*: Pearson Education, 2007. (Addison-Wesley Signature Series). ISBN: 9780321630148. Disponível em: <<https://books.google.com.br/books?id=PV9qfEdv9LOC>>. Citado nas páginas 16–18, 22.

FOWLER, MARTIN. *Continuous Integration*. 2006. Disponível em: <<https://www.martinfowler.com/articles/continuousIntegration.html>>. Acesso em: 5 abr. 2019. Citado nas páginas 16, 17.

GAZETA DO POVO. *Gazeta do Povo. Erro milionário: curitibano tem R\$ 77 milhões depositados por engano em sua conta.* 2019. Disponível em: <<https://www.gazetadopovo.com.br/curitiba/erro-milionario-curitibano-tem-r-77-milhoes-depositados-por-engano-em-sua-conta-0kkiosomhiy8qdf318pgr4qc6>>. Acesso em: 31 jul. 2019. Citado na página 47.

GHAHRAI, AMIR. *The Birth of Automated Testing at Google in 2005*. 2019. Disponível em: <<https://www.testingexcellence.com/problems-test-automation-modern-qa>>. Acesso em: 20 jul. 2019. Citado na página 56.

GITLAB. *Stages of the DevOps Lifecycle*. 2019. Disponível em: <<https://about.gitlab.com/stages-devops-lifecycle/>>. Acesso em: 26 fev. 2019. Citado na página 13.

GIZMODO. *Tesla Autopilot Malfunction Caused Crash That Killed Apple Engineer, Lawsuit Alleges*. 2019. Disponível em: <<https://gizmodo.com/tesla-autopilot-malfunction-caused-crash-that-killed-ap-1834453661>>. Acesso em: 31 jul. 2019. Citado na página 46.

GRADLE. *Building Java Libraries*. Gradle Inc. 2019. Disponível em: <<https://guides.gradle.org/building-java-libraries/>>. Acesso em: 29 abr. 2019. Citado na página 28.

GRAHAM, Dorothy; FEWSTER, Mark. *Experiences of test automation: case studies of software test automation*. [Sine loco]: Addison-Wesley Professional, 2012. Citado nas páginas 49, 51, 52.

GUCKENHEIMER, SAM. *What is Continuous Integration?* 2017. Disponível em: <<https://docs.microsoft.com/en-us/azure/devops/learn/what-is-continuous-integration>>. Acesso em: 7 abr. 2019. Citado nas páginas 16, 17.

HIBBS, Curt; JEWETT, Steve; SULLIVAN, Mike. *The art of lean software development: a practical and incremental approach*. [Sine loco]: " O'Reilly Media, Inc.", 2009. Citado nas páginas 48, 49, 55.

HOZ, PAULA DE LA. *5 must-do security tips for developers*. 2019. Disponível em: <<https://dev.to/terceranexus6/5-must-do-security-tips-for-developers-1a7>>. Acesso em: 29 jul. 2019. Citado na página 32.

JENKINS. *Jenkins Pipeline*. 2019. Disponível em: <<https://jenkins.io/doc/book/pipeline>>. Acesso em: 3 ago. 2019. Citado na página 37.

KIM, GENE. *The Birth of Automated Testing at Google in 2005*. 2017. Disponível em: <<https://dzone.com/articles/case-study-the-birth-of-automated-testing-at-googl>>. Acesso em: 13 jul. 2019. Citado nas páginas 54, 55.

KIM, Gene et al. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press, 2016. (ITpro collection). ISBN: 9781942788072. Disponível em: <<https://books.google.com.br/books?id=ui8hDgAAQBAJ>>. Citado nas páginas 5–9.

LEVESON, Nancy G; TURNER, Clark S. An investigation of the Therac-25 accidents. *Computer, IEEE*, volume 26, número 7, páginas 18–41, 1993. Citado na página 46.

LITTLE, Christopher; HERSCHMANN, Joachim. *Avoid Failure by Developing a Toolchain That Enables DevOps*, out. 2017. Citado nas páginas 13, 14.

NEWHALL, TIA. *Java and Makefiles*. Disponível em: <<https://www.cs.swarthmore.edu/~newhall/unixhelp/javamakefiles.html>>. Acesso em: 28 abr. 2019. Citado nas páginas 22, 23.

NEWMAN, S. *Desmistificando a Lei de Conway*. 2015. Disponível em: <<https://www.thoughtworks.com/pt/insights/blog/demystifying-conways-law>>. Acesso em: 20 fev. 2019. Citado na página 6.

POULSEN, Kevin. Tracking the blackout bug. *Security Focus, April*, 2004. Citado na página 47.

PUPPET. *State of DevOps Report 2018*. 2019. Disponível em: <<https://puppet.com/resources/whitepaper/state-of-devops-report>>. Acesso em: 30 jul. 2019. Citado na página 34.

RED HAT, INC. *O que é DevSecOps?* 2019. Disponível em: <<https://www.redhat.com/pt-br/topics/devops/what-is-devsecops>>. Acesso em: 19 fev. 2019. Citado na página 6.

REHN, A. AND PALMBORG, T. AND BOSTROM, P. *The Continuous Delivery Maturity Model*. 2019. Disponível em: <<https://www.infoq.com/articles/Continuous-Delivery-Maturity-Model>>. Acesso em: 9 jul. 2019. Citado na página 34.

SITEWARE. *O Que É Lean Manufacturing?* 2019. Disponível em: <<https://www.siteware.com.br/processos/o-que-e-lean-manufacturing/>>. Acesso em: 24 fev. 2019. Citado na página 8.

TILLEY, Scott; ROSENBLATT, Harry J. *Systems analysis and design*. [Sine loco]: Cengage Learning, 2017. Citado nas páginas 30, 31.

XMATTERS, ATlassian. *Xmatters Atlassian 2017 Devops Maturity Survey Report*. 2019. Disponível em: <<https://www.xmatters.com/press-release/xmatters-atlassian-2017-devops-maturity-survey-report>>. Acesso em: 3 ago. 2019. Citado na página 36.



Sobre o Professor

 linkedin.com/in/natanlaverde
 github.com/natanlaverde
 lattes.cnpq.br/7686462188972037

Natan de Almeida Laverde

Sou graduado em Ciência da Computação pela Universidade Estadual de Londrina (UEL), tendo participado de diversos projetos de pesquisa e extensão, dentre eles o projeto *Sistema Integrado de Custos Municipais – Módulo Educação*, cujo software foi registrado no Instituto Nacional da Propriedade Industrial (INPI).

Mestre em Ciências da Computação e Matemática Computacional pelo Instituto de Ciências Matemáticas e de Computação (ICMC) da Universidade de São Paulo (USP), com a dissertação *Desenvolvimento de operadores de agrupamento por similaridade em SGBD relacionais*. A pesquisa desenvolvida no mestrado foi apresentada na *10ª Conferência Internacional em Buscas por Similaridade e Aplicações (SISAP) 2017*, pelo artigo *Semantic Similarity Group By Operators for Metric Data*. Durante o mestrado também fiz parte do programa de aperfeiçoamento de ensino, atuando nas disciplinas de *laboratório de bases de dados e mineração a partir de grandes bases de dados*.

Tenho experiência profissionalmente como Analista Desenvolvedor. Trabalhei em uma empresa de desenvolvimento de Software para Gestão Pública, mantendo e desenvolvendo sistemas para os setores de educação, saúde e assistência social. Atualmente trabalho em uma multinacional de serviços de TI, onde atuo no segmento de serviços financeiros, mantendo e desenvolvendo soluções para tratamento de documentos para o setor bancário.



Sobre o Professor

 | [linkedin.com/in/tnobre](https://www.linkedin.com/in/tnobre)

Tiago Nobre

Sou Tecnólogo em Processamento de Dados pela Faculdade de Tecnologia de São Paulo, FATEC. Também sou mestre em Engenharia de Software pela Universidade Federal do Paraná, com ênfase em Testes de Software. Sou desenvolvedor Java, tenho duas certificações, de programador e desenvolvedor Web. Atualmente, trabalho como Consultor de TI na Unimed Curitiba, onde adquiri bastante experiência com BPM. Adoro lecionar.